TRACE

Ghost Data Audit: the complete project guide

HackerRank Orchestrate, October 2026, challenge **“Ghost Data Still Exists”**.

This guide explains every part of the project: the problem, the dataset and its traps, the five-stage pipeline and each module, the policy rules, the website and its features, the evaluation results, the tests, how to run and deploy everything, and what to say in the AI interview.

In one sentence: TRACE checks whether a “deleted” customer’s personal data is really gone. It finds every copy using only evidence, applies the deletion policy at the audit time, and never trusts what the shop’s own workflow claims.

Headline number	Value
Requests audited	518
Verdicts	COMPLETE 153 · INCOMPLETE 355 · NOT_APPLICABLE 10
Shop said COMPLETE but data remains	163 of 254
Steps produced	3,117
Contract check	VALID, 0 errors, 0 uncertain
Sample status accuracy	1.000 (32/32), step F1 0.921
AI model calls / cost	0 / $0: fully deterministic
Run time	about 9 seconds

Repository: github.com/shutupzunaira/TRACE · Generated 5 November 2026

1. The problem

An online shop receives requests from customers to delete their personal data. The shop's deletion workflow reports a status (**COMPLETE**, **PARTIALLY_COMPLETE** or **NOT_STARTED**), but that status can be wrong. Copies can survive in database replicas, logs, search indexes, support tickets, analytics, exports and backups. A sync job or a backup restore can even bring deleted data back.

TRACE is an **auditor**. For each request it establishes what is really true at the case's audit_at time, from evidence alone, and writes one row of `output.csv`:

```
case_id, deletion_status, steps_json
```

deletion_status

Value	Meaning
COMPLETE	Every copy has the required treatment, every retained record is still there, and nothing can recreate the data.
INCOMPLETE	Personal data remains, or a sync or restore can bring it back.
NOT_APPLICABLE	The person did not actually ask for deletion (access, export, correction, opt-out…). steps_json = [].

steps_json: three phases, always in this order

Phase	Operations (state)	evidence_id
FINDING: what is wrong	REPORT_REMAINING_RECORD (PRESENT) · REPORT_EXPOSED_PII (PRESENT) · REPORT_ACTIVE_DATA_FLOW (ACTIVE) · REPORT_RESTORE_RISK (ACTIVE)	Must cite the proof (file / activity / audit id)
REMEDIATION: the fix plan	DISABLE_DATA_FLOW (DISABLED) · BLOCK_RESTORE (BLOCKED) · DELETE_RECORD (ABSENT) · REDACT_RECORD (REDACTED) · ANONYMIZE_RECORD (ANONYMIZED)	Empty: future work
VERIFICATION: final checks	VERIFY_RECORD_ABSENT · VERIFY_RECORD_PRESENT · VERIFY_RECORD_REDACTED · VERIFY_RECORD_ANONYMIZED · VERIFY_DATA_FLOW_DISABLED · VERIFY_RESTORE_BLOCKED	COMPLETE case: cite proof. INCOMPLETE case: empty

Each step has exactly seven fields: `step_no, phase, system_id, resource_id, operation, state, evidence_id`, numbered from 1.

The two golden rules. (1) **Claims are not proof**: a log saying "SUCCESS" never proves data is gone. (2) **Time cutoff**: nothing observed or captured after the case's audit_at may be used.

2. The dataset (dataset/: read-only, never shipped in code.zip)

File	Rows	What it holds
deletion_requests.csv	518	The cases to answer: email, requested_at, reported_status, reported_at, audit_at, request_text (6 languages). Reported: COMPLETE 254, PARTIALLY_COMPLETE 152, NOT_STARTED 112.
sample_deletion_requests.csv + sample_output.csv	32 + 32	Public practice cases with correct answers: used for scoring.
user_identities.csv	850	Current snapshot per user: user_id, name, email, phone, email_hash (= SHA-256 of the lowercase email).
user_audit_log.csv	306	History of identity changes (old → new email / phone).
system_activity.csv	4,011	Global event stream: DELETE 890, UPDATE 672, SYNC 447, INDEX 400, SEND_EMAIL 368, … Results SUCCESS 2,914 / FAILED 721 / PARTIAL 376. All are **claims**, and many rows are about other people.
deletion_rules.csv	15	The deletion policy (section 4.4).

File	Rows	What it holds
evidence_files.csv	94	Manifest: file_id → path, origin, captured_at. It does **not** say which case a file is about.
evidence/	94 files	24 SQL snapshots (primary, read, reporting, regional and failover replicas), 26 logs, JSON / text records, 4 PDFs, 11 screenshots (PNG).

The traps built into the data

- **Old emails:** the request email may be a historical one.
- **Replica drift:** deleted from the primary database but still in a replica.
- **False SUCCESS claims** in the activity log.
- **Recreation:** an enabled sync connector or a restorable backup can bring data back.
- **Late evidence:** evidence captured after audit_at must be ignored.
- **Decoys:** people with the same name, and activity rows about unrelated users.
- **Non-deletion requests** dressed up like deletion cases.

3. Architecture: five stages

Every request in deletion_requests.csv flows through these stages, in order:

Stage	Module(s)	What it does
1. Classify	request_classifier.py	Does the text authorise deletion? If not → NOT_APPLICABLE with []
2. Resolve identity	identity_resolver.py	Current + historical emails, phones and hashes, rolled back to audit_at
3. Find evidence	evidence/ scanner, evidence_search.py	Parse all 94 files once; records and absence proofs at or before audit_at
4. Decide	policy_engine.py, rules.py, activity.py	State of every copy + deletion_rules.csv → decision (resources, flows, restore risks)
4b. Build steps	step_builder.py	Numbered FINDING / REMEDIATION / VERIFICATION → output.csv
5. Explain	audit_db.py → export_web_data.py → web/	Results stored, masked and shown on the Next.js site (display only)

Team role	Owns
P1	Request classifier, identity resolver, audit.db, lineage map, timeline, web export and the website
P2	Evidence scanner and parsers (code/evidence/)
P3	Policy engine, rule book, activity linkage
P4	Step builder and the output contract (CONTRACTS.md, contract-invariant tests)

- **No AI model calls.** Everything is deterministic rules and code: 0 tokens, $0, and byte-identical output on every run (same SHA-256).
- **Each case is isolated** in code/main.py: if one case crashes it is written as INCOMPLETE [] with a warning and the run continues. The final run had 0 errors.
- Stage 5 **never changes an answer**; it only stores and explains results.

4. Each stage in detail

4.1 Stage 1: Classify the request (request_classifier.py, P1)

Regex rules in **English, Spanish, Portuguese, French, Japanese and Korean**, grouped into four signal families:

Family	Examples
DELETION	delete, erase, remove, close my account; eliminar, borrar, excluir, supprimer, effacer, 削除, 삭제
NEGATION	“don’t delete”, “ne supprimez pas”, 削除しないで, 삭제하지 마
KEEP	“keep my account”, “I want to continue using it”
OTHER	access, copy, download / export, correction, marketing opt-out, login help

Signals found	Result
Negation or keep, no deletion term outside the negated span	NOT_APPLICABLE
Negation or keep, plus a separate deletion term	UNCERTAIN (conflict)
Deletion only	IS_DELETION
Deletion plus another intent	UNCERTAIN (mixed)
Only another intent	NOT_APPLICABLE
Nothing matched	UNCERTAIN

- Trap handled: “remove me from your mailing list” is an **opt-out**, not a deletion.
- UNCERTAIN is treated as deletion and flagged for human review.
- Also detects the language (character scripts, stopwords, accented letters) and writes a readable reason.
- **Result:** 508 deletion requests, 10 NOT_APPLICABLE, 0 uncertain. Languages: en 456, pt 14, es 12, fr 12, ja 12, ko 12.

4.2 Stage 2: Resolve the identity (identity_resolver.py, P1)

- Lowercases the email and finds **who owned that email at the request time**. Ownership is time-aware because an email can pass from one user to another.
- Collects every current and historical email, phone and email hash from the audit log, and computes the SHA-256 hash of every old email too.
- **Rolls the identity back to audit_at**: an identifier that only became current after the audit is excluded.
- Returns an IdentityBundle (models.py), or None. If it's None the case is INCOMPLETE, because nothing can be verified.

4.3 Stage 3: Find evidence (code/evidence/ + evidence_search.py, P2)

- The **EvidenceScanner** parses all 94 files **once** into an index. Each case’s query is then a fast lookup.
- One parser per format (evidence/parsers/): sql_snapshots.py (executes the SQL in SQLite), logs.py, json_records.py, pdf_documents.py (pypdf), text_handover.py, images.py.
- Screenshots: text comes from the committed **OCR cache** (evidence/ocr_cache/), produced once with the local RapidOCR engine (an ONNX model, not an LLM). The final run does no OCR.
- Each fact becomes an **ObservedRecord**: system_id, resource_id, presence (PRESENT / ABSENT / UNKNOWN), observed_at, captured_at, copy (replica or table), and **match strength**: STRONG (email, phone, hash, user_id, supplied resource id), LINKED (reached through a confirmed resource) or WEAK (name only). A **claim_only** flag marks second-hand reports.
- **Absence proofs**: a snapshot that lists a whole table or collection without the record. Graded COMPLETE / INFERRED / INCOMPLETE.
- Both cutoffs are applied (observed_at and captured_at ≤ audit_at). The scanner **never decides**: it reports what the evidence shows.

4.4 Stage 4: Decide (policy_engine.py + rules.py + activity.py, P3)

The brain of the system (about 1,000 lines). `decide(identity, evidence, case)` runs five steps:

Step	What happens
1. Seeds	Adds resources that activity explicitly links to the person: data-catalog LINK_RESOURCE_IDENTITY rows, the request's own deletion workflow (WorkflowLinker), and catalog aliases (e.g. a statement number that stands for an export).
2. Validate	Drops naive timestamps and anything after audit_at. Name-only matches are weak and kept only when a stable reference confirms them. Drops records that a strong identifier ties to a different person.
3. Risks	Activity is trusted only for **observed** states: "data flow X remains enabled", "archive X can still be restored", "repopulated downstream resource Y". Never for success claims.
4. Assess	Each copy (primary / replica / table) is judged separately. A copy's state is its **latest authoritative observation**. A same-instant PRESENT/ABSENT conflict resolves to PRESENT. Claims can show a record still exists but **never prove it is gone**. An absence proof counts only if it is complete, error-free, for the same system / category / collection / copy, not contradicted, and newer than the last sighting. Then the category's rule is applied.
5. Status	Any finding → INCOMPLETE, otherwise COMPLETE. In a COMPLETE case, a verification that no evidence proves goes to "unverified" instead of being invented.

The 15 deletion rules (read from the CSV at run time)

Action	Categories and conditions
DELETE	USER_PROFILE, SECONDARY_IDENTIFIER, SEARCH_INDEX_ENTRY, CACHE_ENTRY, USER_EXPORT, MARKETING_PROFILE (always); SUPPORT_ATTACHMENT if it belongs only to the subject
REDACT	SUPPORT_MESSAGE with shared customer / agent / order context; EMAIL_DELIVERY_LOG containing an email; **ACCOUNT_TOMBSTONE**: keep the stable user_id, remove name, email, phone, hash
ANONYMIZE	ANALYTICS_PROFILE and PRODUCT_TELEMETRY still attributable to a user
RETAIN	SECURITY_AUDIT_LOG within retention; LEGAL_HOLD_RECORD with an active hold; BACKUP_ARCHIVE if restore access can be blocked

rules.py sorts each condition sentence into one of three kinds. **TRIGGER** ("still needs work if…"): if not true, the record already has its treatment. **EXCEPTION** ("may be kept if…"): if not true, it falls back to DELETE. **SCOPE** ("only if it belongs only to the subject"): if not true, it is REDACTed. An unrecognised condition sentence **stops the program** instead of guessing.

> **Remediation order (safety first):** disable data flows → block restores → delete → redact → anonymise, each sorted by system, then resource. Stop the recreation paths first, or deleted data comes back.

4.5 Stage 4b: Build the steps (step_builder.py, P4)

Numbers the steps 1…n, enforces the phase order FINDING → REMEDIATION → VERIFICATION, and validates every step against the allowed operation / state pairs. code/validate_output.py re-checks the whole file: 518 rows, sorted, valid JSON, exact fields, consecutive numbers, allowed pairs, empty evidence for future steps, and every cited id exists and was captured at or before that case's audit_at.

4.6 Stage 5: Explain (display only)

- **audit_db.py** saves every case to audit.db (SQLite). Tables: cases, identities, evidence_hits, resources, flows, restore_risks, timeline, lineage_edges, steps. It holds **unmasked** data, so it is git-ignored. Only the real 518-case run writes it.
- **lineage.py** sorts systems into Primary / Copies / Backups, archives & legal holds, adds role labels ("legal hold", "backup", "replica") and builds arrows from activity (labelled as claims).
- **case_timeline.py** builds the time-travel timeline (about 9,100 events in total).
- **export_web_data.py** turns audit.db into **masked** JSON in web/data/: emails → ja****@outlook.com, phones keep the last 2 digits, names → initials, hashes → 8 characters. Masking is also applied inside free text, and a test fails if any dataset email or phone leaks.

5. The website (web/)

Next.js 16, React 19, Tailwind 4, TypeScript, dark theme. It reads only the masked web/data/ files (summary.json, cases.json, lie_detector.json, cases/<id>.json).

Part	What it shows
API routes	/api/summary · /api/cases?status=&q= · /api/cases/[id] · /api/lie-detector
Overview page	Status counts; claim vs reality (163 of 254 “COMPLETE” are really INCOMPLETE); languages; **Lie detector**: per system, how often the shop’s claims were backed by proof
Cases page	Search and filters by status
Case page	Verdict banner (shop’s claim vs TRACE); request text + classifier reason and signals; identity table; identity trail (masked look-alikes labelled “Email 1 / Email 2 (current)”); lineage map with time-travel slider; evidence list; decision per record; steps

Innovation features

Feature	What it does
Replay the ghost	Auto-plays the timeline from start to audit_at with plain-English captions. Claims in amber, proof in red; matching boxes on the map are ringed. Ends with “The shop said: COMPLETE · TRACE found: INCOMPLETE · N records still exposed”. Pause / Restart.
Simulate the fix	INCOMPLETE cases: applies the remediation plan one step per second. Disabled flows go grey, blocked backups get a lock, deleted records fade, redacted / anonymized records turn green. Then a green checklist of verifications and “After this plan: COMPLETE”. Reset. Visual only.
Audit certificate	/cases/[id]/certificate: a printable one-page report (claim vs verdict, decision, evidence ids, plan, audit time) with a **SHA-256 fingerprint** of the case JSON computed in the browser; Print / Save as PDF.

Deploying to Vercel

- Root Directory = web, framework Next.js, no environment variables.
- Vercel cannot run Python, so it serves the committed web/data/ JSON. next.config.ts bundles that data with the API functions (outputFileTracingIncludes).
- To update the site: python code/main.py → python code/export_web_data.py → commit web/data/ → push. Vercel redeploys.

6. Evaluation and results

Scores on the 32 public sample cases (python evaluation/main.py)

Metric	Score
Case-level status accuracy	1.000 (32/32)
Exact case accuracy (status + every step)	0.500 (16/32)
Sequence accuracy	0.500
Step-level accuracy	0.829
Step precision / recall / F1	0.941 / 0.902 / 0.921
Field accuracy: step_no, phase, system, resource, operation, state, evidence	0.959, 0.959, 0.886, 0.829, 0.891, 0.891, 0.953

Why it is not 100%: every miss is one or two verification steps

Root cause	Cases
The expected evidence was captured **after audit_at** (using it would break the time rule)	09, 23, 24, 29, 30
The expected resource appears **nowhere** in participant data	04, 18, 20
Interchangeable anonymous records in one workflow batch: no grounded rule separates them	03, 18/20, 22/23, 25/26, 27/28, 31/32

Full run on 518 cases

Phase	Operation counts
FINDING	REMAINING_RECORD 369 · EXPOSED_PII 214 · ACTIVE_DATA_FLOW 113 · RESTORE_RISK 26
REMEDIATION	DELETE 369 · REDACT 141 · DISABLE_DATA_FLOW 113 · ANONYMIZE 73 · BLOCK_RESTORE 26
VERIFICATION	ABSENT 798 · PRESENT 311 · ANONYMIZED 186 · REDACTED 177 · FLOW_DISABLED 144 · RESTORE_BLOCKED 57

Reported vs reality: of 254 reported COMPLETE, **163 are INCOMPLETE**; of 152 PARTIALLY_COMPLETE, 36 are actually COMPLETE; of 112 NOT_STARTED, 26 are COMPLETE and 10 are not deletion requests.

The 10 custom evaluation cases (tests/test_policy_engine.py, all pass)

#	Case	What it proves
C1	Historical email	An old email is found through the audit log; delete + verify
C2	Identity change after audit	A later email is never used
C3	Replica drift	A primary-copy absence proof does not clear a replica
C4	Claim-only lookup	A ledger NOT_FOUND cannot prove absence; a claim never overrides store state
C5	Recreation risks	Active flows and restore risks are reported; order DISABLE, DISABLE, BLOCK, DELETE
C6	Same-instant conflict	Reported in conflicts, resolved conservatively as PRESENT
C7	Catalog alias	A document carrying only an alias counts as evidence for the linked export
C8	Timezone edge	Included at the equal instant, excluded one second earlier
C9	Tombstone	Empty-PII tombstone is kept and verified REDACTED, never treated as deleted
C10	All 15 rules	A compliant and a non-compliant record for every rule

Improvement history (sample set)

Version	Change	Status / Exact / Step / F1
v0	Placeholder engine (every case INCOMPLETE [])	0.688 / 0.000 / 0.000 / 0.000
v1	First policy engine	1.000 / 0.312 / 0.430 / 0.654
v2	Op-name bug fix; collection-matched absence proofs; cite 404 read-backs first; co-located workflow resources	1.000 / 0.531 / 0.819 / 0.905
v3	Support tickets with shared context → REDACT	1.000 / 0.531 / 0.834 / 0.921
v4	Two-pass workflow attribution (no speculative steps)	1.000 / 0.500 / 0.829 / 0.921
v5	A claim never overrides store state; cite the newest agreeing observation	1.000 / 0.500 / 0.829 / 0.921

Fixed on the real set without labels: case_33 and case_61 (alias resolution), the user_17 alias over-match, case_53 (support screenshot redacted, not deleted), missing **pypdf** silently flipping case_33, and sample runs polluting the website database.

usage_report.md: 0 model calls, 0 tokens, $0 for the pipeline. The Claude Code development sessions (model claude-opus-5-5) are recorded in log.txt. **Tests:** 608 passed, 1 skipped, 1,200 subtests, all in tests/ (python -m pytest from the repo root).

7. Running, files and submission

```
pip install pypdf==6.21.4 pytest==9.2.0 Pillow==13.0.0   # pypdf is REQUIRED
python code/main.py               # 518 requests -> output.csv (+ audit.db), ~9 s
python code/validate_output.py    # must print VALID
python evaluation/main.py         # score the 32 public samples
python code/export_web_data.py    # refresh masked web/data/*.json
cd web && npm install && npm run dev    # http://localhost:3000
```

File / folder	Purpose
AGENTS.md	Rules for AI agents: log every session and turn to log.txt with tool= and model=

File / folder	Purpose
problem_statement.md	The official challenge specification
README.md	Setup, run, website, Vercel deploy, how to extend (new evidence type, new rule)
code/	The pipeline: main.py and every stage; code/evidence/ is the scanner
docs/	CONTRACTS.md (stage interfaces), README_identity.md, P4_CONTRACT_CONFIRMATION.md, this guide
evaluation/	evaluation_report.md, usage_report.md, main.py (sample scorer)
results/	Generated reference outputs (sample_predictions.csv)
scripts/	run_pipeline.sh, run_tests.sh, export_web.sh, build_code_zip.sh
tests/	The whole test suite (pytest.ini at the root)
web/	Next.js website and API routes (Vercel)
log.txt	The chat transcript submitted with the solution

Submit: code.zip (code, README, evaluation reports; without dataset/, node_modules, .next, audit.db, caches, .env or nested zips), the root output.csv, and log.txt at hackerrank.com/contests/hackerrank-orchestrate-october26/challenges/ghost-data-still-exists

Before submitting: rebuild code.zip with sh scripts/build_code_zip.sh after any change, run python code/validate_output.py (must print VALID), and upload code.zip, output.csv and log.txt.

8. Interview cheat sheet

Likely question	Short answer
Why no LLM?	Structured data, exact cutoff and claim rules; deterministic, auditable, reproducible and $0.
How do you avoid trusting claims?	Activity is used only for links and observed risks. Absence must be proven by a complete snapshot of the same copy.
How are old emails handled?	Audit-log history, rolled back to audit_at, with hashes computed for every old email.
What about replicas?	Each copy is judged separately; a primary proof never clears a replica.
Why that remediation order?	Stop flows and restores first, otherwise deleted data is recreated.
What is a tombstone?	Keep the stable user_id, redact name / email / phone / hash; never counted as deleted.
How do you handle time?	Every observation must be timezone-aware, and observed and captured at or before audit_at.
Same-name decoys?	Name-only matches are weak; kept only with a stable reference and never if tied to someone else.
Biggest limitation?	Interchangeable anonymous records in one batch, and expected evidence that only exists after the cutoff.
How was it evaluated?	32 public samples (status 1.000, F1 0.921), 10 custom cases, 608 tests, contract validator.

Appendices: the project's own documents

Each document is reproduced in full from the repository, in this order:

A. Project README: setup, run, deploy, extend `(README.md)`

B. Official problem statement `(problem_statement.md)`

C. Evaluation report `(evaluation/evaluation_report.md)`

D. Usage report (tokens and cost) `(evaluation/usage_report.md)`

E. Stage interfaces (CONTRACTS.md) `(docs/CONTRACTS.md)`

F. P4 contract confirmation `(docs/P4_CONTRACT_CONFIRMATION.md)`

G. Identity and classification notes `(docs/README_identity.md)`

H. Evidence scanner README `(code/evidence/README.md)`

I. AGENTS.md: rules for coding agents `(AGENTS.md)`

Not included: log.txt (the full agent chat log, submitted separately) and the dataset.

Appendix A: Project README: setup, run, deploy, extend

README.md

TRACE: Ghost Data Audit

TRACE checks whether a "deleted" customer's personal data is really gone. For each of 518 deletion requests it decides whether the person actually asked for deletion, finds every current and past identifier, and searches databases, logs, documents, screenshots and backups. It then applies `deletion_rules.csv` to every copy it finds, at the case's audit time, ignoring what workflows claim. The answer is `COMPLETE`, `INCOMPLETE` or `NOT_APPLICABLE`, with an ordered findings → remediation → verification plan, plus a website that explains each case.

Architecture

```
deletion_requests.csv
   |
   ├─ 1. Classify          request_classifier.py   rules in 6 languages: does the text authorise deletion?
   |                                              no -> NOT_APPLICABLE, []
   ├─ 2. Resolve identity  identity_resolver.py    current + historical emails, phones, hashes, as of audit_at
   ├─ 3. Find evidence     evidence/ (scanner)     parses all 94 evidence files once; records, links and
   |                       evidence_search.py      absence proofs available at or before audit_at
   ├─ 4. Decide            policy_engine.py        state of every copy at audit_at + deletion_rules.csv
   |                       rules.py, activity.py   -> decision (resources, flows, restore risks)
   |                       step_builder.py         -> numbered FINDING / REMEDIATION / VERIFICATION steps
   |                                              -> output.csv
   └─ 5. Explain           audit_db.py             every case's results -> audit.db (local, unmasked)
                           lineage.py              data-lineage map (display only, activity = claims)
                           case_timeline.py        time-travel timeline (display only)
                           export_web_data.py      audit.db -> masked JSON in web/data/
                           web/                    Next.js site + API routes (Vercel)
```

`code/main.py` runs stages 1–5 and isolates every case: an error in one case writes `INCOMPLETE []` for that case and the run continues. Stage 5 never changes an answer. Interfaces between stages are in `docs/CONTRACTS.md`; identity and classification are documented in `docs/README_identity.md`.

Repository layout

```
code/          the pipeline: main.py (entry point) and every stage; code/evidence/ is the scanner
dataset/       participant data (read-only input; not in code.zip)
docs/          design notes: CONTRACTS.md, README_identity.md, P4_CONTRACT_CONFIRMATION.md, project guide PDF
evaluation/    evaluation_report.md, usage_report.md, main.py (sample scorer)
results/       generated reference outputs (sample_predictions.csv)
scripts/       shortcuts: run_pipeline.sh, run_tests.sh, export_web.sh, build_code_zip.sh
tests/         the whole test suite (pytest, configured by pytest.ini at the root)
web/           Next.js website and API (deployed on Vercel; reads web/data/)
output.csv     the submitted answers (518 rows)     log.txt   agent chat transcript
```

Setup

Python 3.11+ (developed on 3.12/3.13) and, for the website, Node.js 20.9+. Run every command from the repository root.

```
python -m venv .venv
source .venv/bin/activate              # Windows: .venv\Scripts\activate
pip install pypdf==6.21.4 pytest==9.2.0 Pillow==13.0.0
# optional, only to rebuild the OCR cache: pip install -r code/requirements.txt
```

`pypdf` **is required.** Without it PDF evidence is skipped and at least one answer changes (`case_33`). No API keys or network access are needed: the pipeline makes no model calls, and screenshot text comes from the committed OCR cache (`code/evidence/ocr_cache/`). Output is deterministic.

Run

```
python code/main.py                 # 518 requests -> output.csv (+ audit.db), ~9 s
python code/validate_output.py      # contract check of output.csv; must print VALID
sh scripts/run_pipeline.sh          # the three commands above in one go
python evaluation/main.py           # 32 public samples: run, validate, score -> results/sample_predictions.csv
```

`python code/main.py --requests sample_deletion_requests.csv --output <file>` runs any request file in `dataset/`; only the default 518-case run writes `audit.db`.

Tests:

```
python -m pytest -q                  # whole suite from the repo root (608 passed, 1 skipped)
```

Website (local)

```
python code/main.py && python code/export_web_data.py     # refresh web/data/*.json (masked)
cd web
npm install
npm run dev                                               # http://localhost:3000
```

Pages: **Overview** (status counts, claim vs reality, languages, lie detector by system), **Cases** (search and filters) and **Case** (verdict banner, request and classifier reason, identity and identity trail, lineage map with time-travel slider, evidence, decision per record, steps). API routes: `/api/summary`, `/api/cases?status=&q=`, `/api/cases/[id]`, `/api/lie-detector`.

Privacy: `audit.db` holds unmasked identifiers and is git-ignored. The website only reads `web/data/`, where emails, phones, names and hashes are masked (also inside free text); a test fails if any dataset email or phone appears in the export.

Deploy (Vercel)

Vercel cannot run the Python pipeline, so the site serves the JSON that is committed in `web/data/`.

1. Import the repository in Vercel, **Root Directory** `web`, framework Next.js (defaults for build, install and output), Node.js 24.x, no environment variables.
2. `web/next.config.ts` adds `data/**/*.json` to the API functions (`outputFileTracingIncludes`), so the data is deployed with them; `web/data/` is outside `public/`, so it is only reachable through the API.
3. To publish new results: run `python code/main.py`, then `python code/export_web_data.py`, commit `web/data/` and push. Vercel redeploys automatically.

Extend

Add an evidence type (e.g. `.csv` exports):

1. Write a parser in `code/evidence/parsers/` that returns `ObservedRecord`s (`code/evidence/models.py`). Set `claim_only=True` for second-hand reports (a system saying what it found) and `False` for direct store state.
2. Register its file extension in `_parser_for()` in `code/evidence/parsers/__init__.py`.
3. Add a test in `tests/` with a small real-format file. Nothing else changes: the scanner, policy engine and website pick the new records up.

Add a policy rule:

1. Add a row to `dataset/deletion_rules.csv` (`rule_id, data_category, condition, required_action`).
2. If the condition uses new wording, add a predicate to `_CONDITION_KINDS` in `code/rules.py`. An unrecognised condition raises at start-up instead of being silently ignored.
3. Add a compliant and a non-compliant record for it in `tests/test_policy_engine.py` (`test_each_rule_*`).

Policy engine notes

- Every observation must be timezone-aware, and both its `observed_at` and its file's `captured_at` must be at or before `audit_at`. A name-only match is weak and is used only when a stable reference confirms it; records that a strong identifier ties to another person are dropped.
- Copies (primary, replica, table) are assessed separately. A copy's state is its latest authoritative observation. Ledger claims can show a record or risk still exists, but never prove it is gone.
- Absence proofs must be COMPLETE, error-free, for the same system, category, collection and copy, not contradicted, and newer than the last sighting.
- Activity is used only for catalog links, observed risks (a connector still enabled, an archive still restorable, a job that repopulated data) and workflow attribution. Success messages are never accepted as proof.
- Remediation order: disable data flows → block restores → delete → redact → anonymise, each sorted by `system_id` then `resource_id`. Verification covers the final state of every relevant resource.

Evaluation and submission

- Results, custom test cases and the failures fixed: `evaluation/evaluation_report.md`. Token and cost usage of the final run (no model calls): `evaluation/usage_report.md`.
- Challenge contract: `problem_statement.md`; agent rules and logging: `AGENTS.md`.
- Submit `code.zip` (code, README, evaluation reports; no `dataset/`, `node_modules`, `.next`, `audit.db`, caches or nested archives), the root `output.csv`, and `log.txt` as the chat transcript: https://www.hackerrank.com/contests/hackerrank-orchestrate-october26/challenges/ghost-data-still-exists

Appendix B: Official problem statement

`problem_statement.md`

Ghost Data Still Exists

Northstar Shop (an imaginary company) lost **$2.4 million** after one supposedly deleted customer came back as a ghost in its systems. Every internal dashboard showed the deletion request as **“Complete.”** Weeks later, the same customer tried to create a new account. The platform refused: **“This email is already registered.”**

What looked like a simple signup bug exposed a much larger failure. The investigation uncovered personal data across secondary databases, support attachments, search indexes, analytics exports, logs, archives, and backups. A synchronization job had even recreated information that the primary account service had successfully deleted.

Northstar Shop now needs an AI-powered deletion audit agent that can prove whether a user's personal data is truly gone before another failure causes financial and reputational damage.

For deletion requests, the agent must investigate the supplied records, determine the user's current and historical identifiers, reconstruct what happened across systems, and decide whether the deletion is actually complete.

When data remains, the agent must produce the ordered remediation steps required to remove, redact, or anonymize it. When deletion has already been completed, the agent must produce the checks that prove it. The agent must also preserve records that the supplied policy requires the organization to retain.

Some systems preserve an account tombstone so permitted records can continue to reference the same internal `user_id`. In these records, the stable user reference may remain, but direct identifiers such as name, email, phone, and email hash must be removed.

What You Need to Build

For each request, build a system that determines:

- whether the deletion is `COMPLETE`, `INCOMPLETE`, or `NOT_APPLICABLE` because the person did not request deletion;
- which records or identifiers still expose the user;
- which active processes can recreate deleted data;
- the ordered remediation steps required to complete the request.

A deletion is complete only when:

- every relevant copy has received the treatment required by `deletion_rules.csv`;
- no active sync, import, or restore path can recreate the data;
- retained records contain only the identifiers permitted by the rules; and
- records that must be retained remain available.

Files Provided

All participant-facing files are inside `dataset/`.

File	Contents
`deletion_requests.csv`	Requests that must be audited
`sample_deletion_requests.csv`	Sample requests
`sample_output.csv`	Correct answers for the sample requests
`user_identities.csv`	Current user identities
`user_audit_log.csv`	Historical identity changes
`system_activity.csv`	Deletion, sync, export, backup, restore, indexing, and related operational activity
`deletion_rules.csv`	Required treatment for each data category
`evidence_files.csv`	Manifest of the supplied evidence files
`output.csv`	Blank submission template

Evidence may be stored as PDFs, images, logs, JSON, or SQLite-compatible SQL snapshot exports under `dataset/evidence/`.

Input

Each row in `deletion_requests.csv` is one case:

```
case_id,email,requested_at,reported_status,reported_at,audit_at,request_text
```

`reported_status` may be `COMPLETE`, `PARTIALLY_COMPLETE`, or `NOT_STARTED`. It is a claim from the existing deletion workflow, not proof of the true state.

Required Output

Produce exactly one row in `output.csv` for every case in `deletion_requests.csv`:

```
case_id,deletion_status,steps_json
```

Column	Meaning
`case_id`	Case being audited
`deletion_status`	`COMPLETE`, `INCOMPLETE`, or `NOT_APPLICABLE`
`steps_json`	Ordered JSON array of findings, remediation actions, and verification checks

The final file must contain exactly 518 data rows, sorted by `case_id`.

Each object in `steps_json` must contain:

```
{
  "step_no": 1,
  "phase": "FINDING",
  "system_id": "account_service",
  "resource_id": "account_005",
  "operation": "REPORT_REMAINING_RECORD",
  "state": "PRESENT",
  "evidence_id": "evidence_01"
}
```

`phase` must be `FINDING`, `REMEDIATION`, or `VERIFICATION`. Step numbers start at `1` and increase consecutively. Array order is significant.

For `COMPLETE`, include only verification steps proving the current state. For `INCOMPLETE`, include findings, the complete remediation plan, and the checks that must pass afterward. For `NOT_APPLICABLE`, use an empty JSON array (`[]`) because no deletion or deletion verification is authorized.

A non-deletion row therefore has this form:

```
case_000,NOT_APPLICABLE,[]
```

The grader parses `steps_json` as JSON. JSON object-key order and whitespace are ignored; array order is not.

Allowed Operations

Phase	Operation	State	Meaning
`FINDING`	`REPORT_REMAINING_RECORD`	`PRESENT`	A record that should have been deleted still exists.
`FINDING`	`REPORT_EXPOSED_PII`	`PRESENT`	A record may remain, but it still contains identifiers that should have been redacted or anonymized.
`FINDING`	`REPORT_ACTIVE_DATA_FLOW`	`ACTIVE`	An active process can copy or recreate the user's data.
`FINDING`	`REPORT_RESTORE_RISK`	`ACTIVE`	A backup or restore path can bring deleted data back.
`REMEDIATION`	`DISABLE_DATA_FLOW`	`DISABLED`	Stop the process that can copy or recreate the data.
`REMEDIATION`	`BLOCK_RESTORE`	`BLOCKED`	Prevent the affected backup from restoring the user's data.
`REMEDIATION`	`DELETE_RECORD`	`ABSENT`	Remove the entire record.
`REMEDIATION`	`REDACT_RECORD`	`REDACTED`	Keep the record but remove the identifying fields required by policy.
`REMEDIATION`	`ANONYMIZE_RECORD`	`ANONYMIZED`	Keep the record after irreversibly unlinking it from the user.
`VERIFICATION`	`VERIFY_RECORD_ABSENT`	`ABSENT`	Confirm that a record no longer exists.
`VERIFICATION`	`VERIFY_RECORD_PRESENT`	`PRESENT`	Confirm that a policy-required retained record still exists.
`VERIFICATION`	`VERIFY_RECORD_REDACTED`	`REDACTED`	Confirm that required identifying fields have been removed.
`VERIFICATION`	`VERIFY_RECORD_ANONYMIZED`	`ANONYMIZED`	Confirm that the record can no longer be linked to the user.
`VERIFICATION`	`VERIFY_DATA_FLOW_DISABLED`	`DISABLED`	Confirm that the relevant data flow is inactive.
`VERIFICATION`	`VERIFY_RESTORE_BLOCKED`	`BLOCKED`	Confirm that the affected data cannot be restored.

Rules

- Resolve the user's current and historical identifiers.
- Determine the resource state at `audit_at` using the available evidence. Reported workflow statuses and operation results are claims.
- Apply `deletion_rules.csv` to every relevant record, including account tombstones, active data flows, and restore risks.
- Order steps by phase: findings, remediation, then verification. Within each phase, follow the operation order in the table above, followed by `system_id` and `resource_id` in ascending order.
- Findings and verification steps for `COMPLETE` cases must cite a `file_id`, `activity_id`, or `audit_id`. Proposed remediation and future verification steps for `INCOMPLETE` cases use an empty `evidence_id`.

Evaluation

`output.csv` will be compared with hidden canonical answers. Scoring will consider the correctness of `deletion_status`, identity resolution, discovered records and risks, cited evidence, policy decisions, and the completeness and order of `steps_json`.

Your `code.zip` must include `evaluation/evaluation_report.md` containing:

- case-level status accuracy;
- step-level and field-level accuracy;
- sequence accuracy;
- at least three additional evaluation cases created by your team; and
- failures found and improvements made.

Submission

Item	Description
`code.zip`	Runnable solution, prompts, configuration, README, and `evaluation/` folder
`output.csv`	One answer row for every request
`chat_transcript`	Transcript showing how the system was developed or used
AI interview	Explanation of the approach and evaluation

Token Usage and Cost Analysis

Your `code.zip` must include one token-usage file:

```
evaluation/usage_report.md
```

The report must summarize the final full-dataset run that produced `output.csv`, including model providers and names, model calls, input and output tokens, total and average tokens per request, and estimated total and per-request cost. If multiple models are used, include both per-model and overall totals in the same file.

Do not include API keys, credentials, or sensitive configuration values in the submission.

These are the required deliverables. Participants are encouraged to improve retrieval, multimodal interpretation, identity and deletion-state reconstruction, remediation planning, deterministic verification, batching, caching, and token efficiency.

Appendix C: Evaluation report

`evaluation/evaluation_report.md`

Evaluation Report — Ghost Data Still Exists (TRACE)

Reproduce every number in this report with:

```
python evaluation/main.py                       # pipeline on the 32 public samples + scoring
python -m pytest -q                             # whole suite from the repo root: 608 passed, 1 skipped, 1,200 su
  btests
python code/main.py && python code/validate_output.py   # full 518-case run + contract check
```

Re-verified on 2026-11-05: the sample metrics below are unchanged, `validate_output.py` reports VALID, and the full run gives COMPLETE 153 / INCOMPLETE 355 / NOT_APPLICABLE 10.

The solution is fully deterministic (no model calls, no randomness). Two consecutive full runs produce byte-identical `output.csv` files (verified by SHA-256).

1. What was measured

The only labelled data available to participants is `dataset/sample_output.csv` (32 cases). It was used during development, so these are **in-sample** numbers and are optimistic. The 518 evaluation requests have no public labels; for them we report contract validity and the distribution of decisions (section 4).

Metric definitions (`code/evaluate.py`; JSON key order and whitespace ignored, array order kept):

Metric	Definition
Case-level status accuracy	`deletion_status` equal
Exact case accuracy	status **and** the complete step list equal
Step-level (row) accuracy	position-wise: step i equal in all 7 fields, over `max(len(pred), len(expected))` positions
Step precision / recall / F1	order-insensitive multiset match of steps (all fields except `step_no`)
Field-level accuracy	position-wise per field, over `max(len(pred), len(expected))` positions
Sequence accuracy	ordered list of `(phase, system_id, resource_id, operation)` identical

2. Results on the 32 public sample cases (final version)

Metric	Value
Case-level deletion-status accuracy	**1.000** (32/32)
Exact case accuracy	0.500 (16/32)
Sequence accuracy	0.500 (16/32)
Step-level (row) accuracy	0.829
Step precision	0.941
Step recall	0.902
Step F1	0.921

Field-level accuracy:

step_no	phase	system_id	resource_id	operation	state	evidence_id
0.959	0.959	0.886	0.829	0.891	0.891	0.953

Status by class: 10/10 COMPLETE, 22/22 INCOMPLETE (the sample set contains no NOT_APPLICABLE case; that path is covered by tests and by the 10 non-deletion requests in the evaluation set).

Remaining differences, by root cause

All 16 non-exact cases have the correct status; each differs only in one or two verification steps.

Root cause	Cases	Why it cannot be fixed with participant data
Expected step's only evidence was **captured after** `audit_at`	09, 23, 24, 29, 30	e.g. `audit_record_4128` appears only in `evidence_02` (captured 2026-09-17) for an audit at 2026-09-15; `audit_record_4902` only in `evidence_06` (captured 09-18) for an audit at 09-16. Using them would break the cutoff rule (AGENTS.md §6.2/§6.6).

Root cause	Cases	Why it cannot be fixed with participant data
Expected resource appears **nowhere** in participant data	04 (`account_004`), 18 (`audit_record_6645`), 20 (`audit_record_5569`)	Not in any evidence file, activity row or audit row.
Interchangeable de-identified records in one workflow batch	03, 18/20, 22/23, 25/26, 27/28, 31/32	Two or three requests share the same workflow window and `reported_at`; the deleted accounts (bare 404 lookups) and anonymised analytics rows carry no identifier. Activity order is not consistent across batches (08-30: reverse order, 09-02: same order), so no grounded rule separates them.

3. Additional evaluation cases created by the participant

Each case is an isolated, synthetic world (real-format evidence files parsed by the production P2 scanner, plus test identity / audit / activity tables) in `tests/test_policy_engine.py`. All pass.

#	Case (test name)	Input	Expected	Result
C1	Historical email (`test_historical_email_matches_evidence`)	Subject changed email in May; a search document still holds the old email	Old email resolved through `user_audit_log`; `REPORT_REMAINING_RECORD` → `DELETE_RECORD` → `VERIFY_RECORD_ABSENT`; `audit_01` in identity history	PASS
C2	Identity change after the audit (`test_identity_change_after_audit_at_is_not_used`)	Email changed after `audit_at`; evidence uses the later email	Later email not used, record not attributed	PASS
C3	Replica drift (`test_primary_absent_replica_present_is_remaining`, `test_primary_copy_proof_does_not_prove_replica_absence`)	Account absent from the primary table but present in the read replica; a later primary-only listing omits it	Remaining record reported from the replica copy; a primary-copy absence proof is not accepted for the replica	PASS
C4	Claim-only lookup (`test_claim_only_not_found_does_not_prove_absence`, `test_claim_never_overrides_authoritative_store_state`)	Only a ledger `NOT_FOUND` claim for the account; separately, a store 404 followed by a newer `FOUND` claim	Claim cannot prove absence (no evidence-backed verification, listed in `unverified`); a claim never overrides store state	PASS
C5	Recreation risks and safe order (`test_active_sync_and_restore_risks_and_safe_order`)	Connector observed disabled, then a status check confirms it still enabled; a second connector repopulated the subject's search entry; an archive check confirms restore access	Both flows `REPORT_ACTIVE_DATA_FLOW` (citing the activity ids), `REPORT_RESTORE_RISK`; remediation `DISABLE, DISABLE, BLOCK, DELETE`; every remediated item verified	PASS
C6	Same-instant conflict (`test_same_instant_authoritative_conflict_is_reported_and_conservative`)	Two authoritative observations at the same instant: PRESENT and ABSENT	Conflict reported in `decision["conflicts"]`; resolved conservatively as PRESENT	PASS
C7	Catalog alias (`test_catalog_alias_links_document_to_resource`)	Catalog links `export_9` (alias `NST-2026-0901-9`) to the subject; a statement document carries only the alias	Document treated as evidence of `export_9`; remaining export reported	PASS
C8	Timezone edge (`test_equivalent_instants_with_different_offsets`)	Record observed at 12:00Z, audit at 14:00+02:00 and at 13:59:59+02:00	Included at the equal instant, excluded one second earlier	PASS
C9	Tombstone (`test_tombstone_with_empty_pii_is_still_present_not_deleted`)	Tombstone with all direct identifiers null	Kept and `VERIFY_RECORD_REDACTED` with evidence; never treated as deleted	PASS
C10	All 15 rules (`test_each_rule_noncompliant_record`, `test_each_rule_compliant_record`)	One non-compliant and (where meaningful) one compliant record per CSV rule	Correct finding / remediation / verification per rule, rule id recorded	PASS

Full test inventory (`python -m pytest` from the repo root, after the files moved to `tests/`): 608 passed, 1 skipped, and 1,200 subtests passed. This includes the 32 P3-P4 contract invariant checks and the website-data suites.

4. Full evaluation set (518 requests, final run)

- Contract check (`code/validate_output.py`): **VALID**. 518 rows, no missing or duplicate case ids, sorted, valid JSON arrays, exact step fields, consecutive numbering, phase order, allowed operation/state pairs, empty evidence for future steps, every cited id exists and was captured at or before that case's `audit_at`.
- Errors during the run: 0. Uncertain classifications: 0.
- Decisions: COMPLETE 153, INCOMPLETE 355, NOT_APPLICABLE 10. Steps: 3,117.

Phase	Operation	Count
FINDING	REPORT_REMAINING_RECORD	369
FINDING	REPORT_EXPOSED_PII	214
FINDING	REPORT_ACTIVE_DATA_FLOW	113
FINDING	REPORT_RESTORE_RISK	26
REMEDIATION	DELETE_RECORD	369
REMEDIATION	REDACT_RECORD	141
REMEDIATION	DISABLE_DATA_FLOW	113
REMEDIATION	ANONYMIZE_RECORD	73
REMEDIATION	BLOCK_RESTORE	26
VERIFICATION	VERIFY_RECORD_ABSENT	798
VERIFICATION	VERIFY_RECORD_PRESENT	311
VERIFICATION	VERIFY_RECORD_ANONYMIZED	186
VERIFICATION	VERIFY_RECORD_REDACTED	177
VERIFICATION	VERIFY_DATA_FLOW_DISABLED	144
VERIFICATION	VERIFY_RESTORE_BLOCKED	57

Reported workflow status compared with our decision: of 254 requests reported `COMPLETE`, 163 are INCOMPLETE at `audit_at`; of 152 reported `PARTIALLY_COMPLETE`, 36 are in fact COMPLETE; of 112 reported `NOT_STARTED`, 26 are COMPLETE and 10 are not deletion requests.

5. Failures found and improvements made

Sample-set metrics after each change (status / exact / step-level / step F1):

Version	Change	Status	Exact	Step-level	F1
v0	Integrated P1+P2 with the placeholder policy engine (every case `INCOMPLETE []`)	0.688	0.000	0.000	0.000
v1	First policy engine	1.000	0.312	0.430	0.654
v2	Fixed a verification-operation name bug that crashed 5 cases; absence proofs must match the observed **collection** (a listing of another store was overriding a PRESENT security record); cite a direct 404 read-back before a snapshot omission; prefer workflow resources **co-located** in the subject's own evidence files	1.000	0.531	0.819	0.905
v3	Support tickets with a requester block are shared conversation context → REDACT, not DELETE	1.000	0.531	0.834	0.921
v4	Two-pass workflow attribution: anonymised analytics rows only to requests that are otherwise compliant (removes speculative steps from incomplete requests)	1.000	0.500	0.829	0.921
v5	Stricter rule: a claim never overrides store state (dropped to 0.438 / F1 0.910 because findings then cited an older check); fixed by citing the newest observation that agrees with the store-decided state	1.000	0.500	0.829	0.921

Failures found on the **evaluation** set by inspecting decisions (no labels used) and fixed:

- `case_33`, `case_61` were COMPLETE with no steps. The catalog links resources by alias (a statement reference `NST-2026-0718-1842`, a file name `delivery_confirmation_19.pdf`). Alias resolution re-keys such evidence to the linked resource, so both are now evidence-backed INCOMPLETE findings.
- A tombstone alias equal to the user id (`user_17`) matched every record mentioning that user and invented a PII field. Alias matching now requires the record to be the artefact (same id, document reference or file name).
- `case_53`: a support-ticket screenshot (agent conversation with an order number) was deleted; the OCR text is now checked for conversation/order context, so it is redacted.

Found during the final check (2026-11-05):

- **Missing `pypdf` silently changes an answer.** In a Python environment without `pypdf`, PDF evidence is skipped and `case_33` flips from INCOMPLETE (export_1842, cited from the PDF `evidence_18`) to COMPLETE with no steps. The README now lists `pypdf` as required, and the final run was made with `pypdf==6.21.4` installed.
- **Sample runs polluted the website data.** `evaluation/main.py` also wrote the 32 sample cases into `audit.db`. `main.py` now writes `audit.db` only for the real 518-case run.

Changes made to teammates' code: none to P1/P2 logic. In P1's `test_main.py`, two assertions that encoded placeholder output (`("INCOMPLETE", "[]")`) were replaced by the test's stated intent (the case runs without error and yields a contract status; a naive `audit_at` equals the same UTC instant).

Appendix D: Usage report (tokens and cost)

evaluation/usage_report.md

Usage Report — final full-dataset run

Run that produced the submitted `output.csv`: `python code/main.py` over the 518 requests in `dataset/deletion_requests.csv`, re-run on 2026-11-05 (wall time about 9 seconds on a laptop, including the audit.db write for the website; 0 failed cases). Result: COMPLETE 153, INCOMPLETE 355, NOT_APPLICABLE 10, byte-identical to the 2026-11-04 run.

Model usage

The final pipeline makes **no language-model calls**. Every stage is deterministic code:

Stage	Owner	Technique	Model calls
Request classification	P1 (`request_classifier.py`)	multilingual keyword / pattern rules	0
Identity resolution	P1 (`identity_resolver.py`)	current snapshot + audit-log roll-back to `audit_at`	0
Evidence discovery	P2 (`code/evidence/`)	parsers for SQL, JSON, logs, PDF text (pypdf), text; screenshot text from the committed OCR cache (`ocr_mode="cache_only"`)	0
Policy and evidence validation	P3 (`policy_engine.py`, `rules.py`, `activity.py`)	rule book loaded from `deletion_rules.csv`, evidence-state reconstruction	0
Step building	`step_builder.py`	numbering and contract validation	0
Website data (display only)	P1 (`audit_db.py`, `lineage.py`, `case_timeline.py`, `export_web_data.py`)	SQLite + static JSON export, masking	0

Screenshot text was produced earlier by P2 with the local RapidOCR engine (an ONNX OCR model, not a language model, no API and no tokens) and committed to `code/evidence/ocr_cache/`. The final run reads that cache only and runs no OCR.

Totals

Provider	Model	Calls	Input tokens	Output tokens	Total tokens	Avg tokens / request	Est. cost	Est. cost / request
— (none)	—	0	0	0	0	0	$0.00	$0.00
Overall		**0**	**0**	**0**	**0**	**0**	**$0.00**	**$0.00**

Requests processed: 518 (508 deletion requests assessed, 10 NOT_APPLICABLE).

Not included above, because it was not part of the pipeline run: the AI coding assistant used to write the code (Claude Code, `claude-opus-5-5`). The harness exposed no token or cost counters for those development sessions, so they are not estimated here; the sessions are recorded in the root `log.txt`.

Appendix E: Stage interfaces (CONTRACTS.md)

docs/CONTRACTS.md

Team contracts: P1 → P2 → P3 → P4

How data moves between the pipeline stages in `code/main.py`:

```
classify_request(text)                          P1  request_classifier.py
IdentityResolver.resolve(...) -> identity       P1  identity_resolver.py    (see README_identity.md)
find_records(identity, case) -> evidence        P2  evidence_search.py      section 1
decide(identity, evidence, case) -> decision P3  policy_engine.py       section 2 (PROPOSAL)
build_steps(decision) -> steps                  P4  step_builder.py         section 3 (PROPOSAL)
```

`main.py` checks each hand-over. If a stage raises an error or returns the wrong type, that case is written as `INCOMPLETE, []` and listed at the end of the run, so one bug never stops `output.csv` from being written.

All times are aware UTC `datetime`s. Parse CSV/evidence strings with `utils.parse_ts()`. Never use anything captured after the case's `audit_at`.

1. P2 → P3: `find_records(identity, case) -> dict`

Input. `main.py` passes a copy of the CSV row in which `case["audit_at"]` is already an aware UTC `datetime` (from `utils.parse_ts()`), never the raw string. A CSV value without a timezone becomes UTC. If `audit_at` is empty, `main.py` writes `INCOMPLETE, []` and does **not** call `find_records()` at all, so P2 never has to search without a cut-off. The other fields stay strings.

Status: agreed shape, enforced by `main.py`**.** The keys live in `models.EVIDENCE_KEYS`, and `models.empty_evidence()` returns an empty example. A missing key makes the case fail (`INCOMPLETE, []`). An extra key only prints a warning.

Key	Type	Meaning	From P2's `EvidenceScanner`
`records`	`list[ObservedRecord]`	Every observation linked to the subject and available at `audit_at`, claims included, nothing collapsed	`find_records(subject, audit_at)`
`latest_by_resource`	`dict[(system_id, resource_id, replica_or_table), ObservedRecord]`	Latest **authoritative** observation per copy. Replicas are separate keys	`latest_by_resource(subject, audit_at)`
`referenced_resources`	`list[dict]`	Resources tied to the subject **without** a direct identifier, e.g. an `analytics_9406` that only `system_activity.csv` links to the user. Each item: `{system_id, resource_id, via, source_id}`, where `via` is `"system_activity"` or `"linked_record"` and `source_id` is the `activity_id` / `file_id` that links it	`records_for_resource(...)` for these
`absence_proofs`	`dict[(system_id, resource_id), list[AbsenceProof]]`	Complete snapshots (at or before `audit_at`) that prove a resource is **not** there, newest first. Only `COMPLETE` proofs	`absence_proofs(system_id, resource_id, audit_at)`
`identifier_provenance`	`dict[str, list[str]]`	Which subject identifier matched in which evidence: `"email:x@y.com" -> ["evidence_12", ...]`. Keys use P2's `kind:value` terms	from `ObservedRecord.identifiers_matched`

How P2 builds `subject` from P1's bundle: `SubjectIdentifiers(**identity.as_subject_kwargs())` (see README_identity.md, "For P2 and P3 – adapter").

Placeholder today (`evidence_search.py`): all five keys present, all empty.

2. P3 → P4: `decide(identity, evidence, case) -> dict`

PROPOSAL, P3 and P4 to confirm.

Today `main.py` enforces only `deletion_status`. The other keys are a proposal derived from `problem_statement.md` and the 32 sample answers.

```
{
```

```
  "deletion_status": "COMPLETE" | "INCOMPLETE",
  "resources":      [ {system_id, resource_id, category, required_treatment,
                       state_at_audit, satisfied, evidence_ids} ],
  "flows":          [ {system_id, resource_id, kind, active_at_audit, evidence_id} ],
  "restore_risks":  [ {system_id, resource_id, active_at_audit, evidence_id} ],
  "timeline":       [ {time, system_id, resource_id, event, state_after, evidence_id} ],
}
```

`resources` — one per relevant record

Field	Type	Meaning
`system_id`	`str`	e.g. `account_service`
`resource_id`	`str`	e.g. `account_005`
`category`	`str`	`data_category` from `deletion_rules.csv`, e.g. `USER_PROFILE`
`required_treatment`	`str`	`DELETE` / `REDACT` / `ANONYMIZE` / `RETAIN` / `TOMBSTONE` (from the rule's `required_action`; `ACCOUNT_TOMBSTONE` is `TOMBSTONE`)
`state_at_audit`	`str`	`PRESENT` / `ABSENT` / `REDACTED` / `ANONYMIZED` / `UNKNOWN`, the true state at `audit_at` from evidence, not from claims
`satisfied`	`bool`	`True` if `state_at_audit` already meets `required_treatment`
`evidence_ids`	`list[str]`	`file_id` / `activity_id` / `audit_id` proving `state_at_audit`, best first. P4 cites `evidence_ids[0]`

`flows` — processes that can copy or recreate data (sync, import, export, indexing)

Field	Type	Meaning
`system_id`, `resource_id`	`str`	the flow / connector
`kind`	`str`	e.g. `SYNC`, `IMPORT`, `EXPORT`, `INDEX`
`active_at_audit`	`bool`	can it still run at `audit_at`?
`evidence_id`	`str`	proof of that state

`restore_risks` — backups that could bring data back

Same as `flows` without `kind`. `active_at_audit=True` means restore is not blocked.

`timeline` — what happened, oldest first

`time` (ISO-8601 UTC string), `system_id`, `resource_id`, `event` (e.g. `DELETE`, `SYNC`, `RESTORE`), `state_after`, `evidence_id`. P4 does not need it to build steps. It is for debugging and the evaluation report.

`deletion_status` should be `COMPLETE` only if every resource is `satisfied`, no flow and no restore risk is `active_at_audit`, and every `RETAIN` record is `PRESENT`.

3. P4: turning the decision into `steps_json`

PROPOSAL, P3 and P4 to confirm.

Operation for each item

Item	Finding (only if not satisfied / active)	Remediation	Verification
resource, `DELETE`	`REPORT_REMAINING_RECORD` / `PRESENT`	`DELETE_RECORD` / `ABSENT`	`VERIFY_RECORD_ABSENT` / `ABSENT`
resource, `REDACT`	`REPORT_EXPOSED_PII` / `PRESENT`	`REDACT_RECORD` / `REDACTED`	`VERIFY_RECORD_REDACTED` / `REDACTED`
resource, `TOMBSTONE`	`REPORT_EXPOSED_PII` / `PRESENT`	`REDACT_RECORD` / `REDACTED` (keep `user_id` only)	`VERIFY_RECORD_REDACTED` / `REDACTED`
resource, `ANONYMIZE`	`REPORT_EXPOSED_PII` / `PRESENT`	`ANONYMIZE_RECORD` / `ANONYMIZED`	`VERIFY_RECORD_ANONYMIZED` / `ANONYMIZED`
resource, `RETAIN`	none	none	`VERIFY_RECORD_PRESENT` / `PRESENT`
flow, active	`REPORT_ACTIVE_DATA_FLOW` / `ACTIVE`	`DISABLE_DATA_FLOW` / `DISABLED`	`VERIFY_DATA_FLOW_DISABLED` / `DISABLED`
restore risk, active	`REPORT_RESTORE_RISK` / `ACTIVE`	`BLOCK_RESTORE` / `BLOCKED`	`VERIFY_RESTORE_BLOCKED` / `BLOCKED`

Which steps to write

- `INCOMPLETE`:
  - FINDING + REMEDIATION for every unsatisfied resource and every active flow or restore risk.
  - VERIFICATION for every resource (satisfied or not), and for every flow or restore risk that was remediated.
  - Findings cite `evidence_ids[0]` / `evidence_id`. Remediation and verification use `""`.
- `COMPLETE`: VERIFICATION only, one per resource, flow and restore risk, each citing its evidence.

The 32 sample answers support this:

- In `INCOMPLETE` samples, 19 `VERIFY_RECORD_ABSENT` steps belong to records that were already deleted and have no finding.
- Every flow, restore, redact and anonymize verification in an `INCOMPLETE` sample has a matching remediation.
- `COMPLETE` samples verify disabled flows (3) and blocked restores (3) with evidence.
- Every finding cites evidence. No remediation step does, and no `INCOMPLETE` verification step does.

Order

Sort all steps by **(position of the operation in the table in** `problem_statement.md`**,** `system_id`**,** `resource_id`**)**, then number them from 1. The operation table order is:

```
REPORT_REMAINING_RECORD, REPORT_EXPOSED_PII, REPORT_ACTIVE_DATA_FLOW, REPORT_RESTORE_RISK,
DISABLE_DATA_FLOW, BLOCK_RESTORE, DELETE_RECORD, REDACT_RECORD, ANONYMIZE_RECORD,
VERIFY_RECORD_ABSENT, VERIFY_RECORD_PRESENT, VERIFY_RECORD_REDACTED, VERIFY_RECORD_ANONYMIZED,
VERIFY_DATA_FLOW_DISABLED, VERIFY_RESTORE_BLOCKED
```

This puts findings first, then remediation, then verification, and it disables flows and blocks restores before records are changed. All 32 sample answers are sorted exactly this way.

Example: sample case_06

`decide()` would return (only the relevant parts):

```
{
  "deletion_status": "INCOMPLETE",
  "resources": [
    {"system_id": "account_service", "resource_id": "account_006", "category": "USER_PROFILE",
     "required_treatment": "DELETE", "state_at_audit": "ABSENT", "satisfied": True,
     "evidence_ids": ["<file_id that proves account_006 is absent>"]},
    {"system_id": "search_service", "resource_id": "search_001", "category": "SEARCH_INDEX_ENTRY",
     "required_treatment": "DELETE", "state_at_audit": "PRESENT", "satisfied": False,
     "evidence_ids": ["evidence_57"]},
  ],
  "flows": [], "restore_risks": [], "timeline": [...],
}
```

`search_001` is `DELETE` with `satisfied=False`, so it becomes `REPORT_REMAINING_RECORD → DELETE_RECORD → VERIFY_RECORD_ABSENT`. `account_006` is already satisfied, so it gets only `VERIFY_RECORD_ABSENT`. After sorting, `build_steps()` returns exactly the sample answer:

```
[
  {"step_no": 1, "phase": "FINDING",      "system_id": "search_service",  "resource_id": "search_001",  "operati
  on": "REPORT_REMAINING_RECORD", "state": "PRESENT", "evidence_id": "evidence_57"},
  {"step_no": 2, "phase": "REMEDIATION",  "system_id": "search_service",  "resource_id": "search_001",  "operati
  on": "DELETE_RECORD",           "state": "ABSENT",  "evidence_id": ""},
  {"step_no": 3, "phase": "VERIFICATION", "system_id": "account_service", "resource_id": "account_006", "operati
  on": "VERIFY_RECORD_ABSENT",    "state": "ABSENT",  "evidence_id": ""},
  {"step_no": 4, "phase": "VERIFICATION", "system_id": "search_service",  "resource_id": "search_001",  "operati
  on": "VERIFY_RECORD_ABSENT",    "state": "ABSENT",  "evidence_id": ""}
]
```

Note that step 3 (`account_service`) comes before step 4 (`search_service`): within `VERIFY_RECORD_ABSENT`, steps are sorted by `system_id`.

4. Open questions

1 **Who reads** `system_activity.csv`**?** P2's scanner reads only the evidence files. Flows, restore risks, `activity_id` citations and `referenced_resources` need the activity stream. P2 or P3 should own it.

2 **Replicas.** P2 keys `latest_by_resource` by `replica_or_table`, but a step has no replica field. Does P3 merge all copies of one `resource_id` into one resource (unsatisfied if any copy is), or keep them apart?

3 `RETAIN` **record missing at** `audit_at`**.** No allowed operation can fix that. Should it make the case `INCOMPLETE` with only a verification step?

4 **Extra fields.** Should `resources` also carry `rule_id` (to explain the decision in the evaluation report) and `replica_or_table`?

Appendix F: P4 contract confirmation

docs/P4_CONTRACT_CONFIRMATION.md

P3 → P4 contract confirmation (P4)

Status: P3 CONFIRMATION COMPLETE. P3 reviewed commit `e72909a` and approved the implementation and tests. One item is still an **OPEN TEAM QUESTION**: a retained record missing at `audit_at` (§6 item 5).

This document records what the code does today (`policy_engine.decide()` → `step_builder.build_steps()`) and how that compares with the proposal in `CONTRACTS.md` sections 2 and 3. Those sections are still marked "PROPOSAL, P3 and P4 to confirm" because `CONTRACTS.md` has not been changed yet.

Decisions confirmed by P3

1. `decide()` **owns all policy decisions.** It decides which records, flows and restore paths are relevant, which operation each one gets, the `deletion_status`, the evidence cited, and the order inside each phase.
2. `build_steps()` **only constructs, validates and numbers steps.** It must not make policy decisions or independently reorder policy results.
3. **Numeric natural sorting of** `resource_id` **is confirmed.** For example, `account_9` comes before `account_10`.
4. `decide()` **returns only** `COMPLETE` **or** `INCOMPLETE`**.** `NOT_APPLICABLE` comes from P1's classifier in `main.py`. `decide()` is not called for those requests.
5. **Every** `INCOMPLETE` **case has at least one finding.**

Checked against: `AGENTS.md` §6.3–6.6, `code/CONTRACTS.md`, `code/policy_engine.py`, `code/step_builder.py`, `code/main.py`, `code/validate_output.py`, `code/tests/test_policy_engine.py`, `code/test_main.py`. Implementation at commit `cafd103`.

1. Input and output

The implemented interface is **not** the one in the `CONTRACTS.md` §2 proposal.

	`CONTRACTS.md` **proposal**	**Implemented**
Who chooses operations	P4 maps `resources` / `flows` / `restore_risks` to operations	P3 (`decide()`) chooses every operation
Who orders steps	P4 sorts	P3 orders inside each phase (`_ordered()`)
What P4 reads	`resources`, `flows`, `restore_risks`	`deletion_status`, `findings`, `remediation`, `verification`
What P4 does	Maps, filters, sorts, numbers	Validates, fills in `evidence_id` rules, numbers

`decide()` returns a dict. `build_steps()` reads only these keys:

```
{
  "deletion_status": "COMPLETE" | "INCOMPLETE",   # NOT_APPLICABLE is also accepted
  "findings":     [{system_id, resource_id, operation, state, evidence_id}, ...],
  "remediation":  [...same shape...],
  "verification": [...same shape...],
}
```

The other keys (`resources`, `flows`, `restore_risks`, `timeline`, `conflicts`, `unverified`, `identity`, `notes`, `case_id`) are for the audit trail and the evaluation report. P4 ignores them. Their shapes differ from the proposal. For example, `resources` items carry `rule_id`, `action`, `finding`, `remediation`, `verification` and `copies` instead of `required_treatment`, `state_at_audit` and `satisfied`. `timeline` is always `[]`.

`build_steps()` returns `list[dict]`. Each dict has exactly the AGENTS.md §6.3 fields: `step_no, phase, system_id, resource_id, operation, state, evidence_id`. `main.py` serialises the list with `json.dumps(..., ensure_ascii=False, separators=(",", ":"))`.

2. Operation and state mappings

P3 sets the operation. `build_steps()` checks that it belongs to the phase and that its state matches. If `state` is missing, it fills it in. If the state is wrong, it raises `ValueError`.

Phase	Operation → required state
FINDING	`REPORT_REMAINING_RECORD→PRESENT`, `REPORT_EXPOSED_PII→PRESENT`, `REPORT_ACTIVE_DATA_FLOW→ACTIVE`, `REPORT_RESTORE_RISK→ACTIVE`
REMEDIATION	`DISABLE_DATA_FLOW→DISABLED`, `BLOCK_RESTORE→BLOCKED`, `DELETE_RECORD→ABSENT`, `REDACT_RECORD→REDACTED`, `ANONYMIZE_RECORD→ANONYMIZED`
VERIFICATION	`VERIFY_RECORD_ABSENT→ABSENT`, `VERIFY_RECORD_PRESENT→PRESENT`, `VERIFY_RECORD_REDACTED→REDACTED`, `VERIFY_RECORD_ANONYMIZED→ANONYMIZED`, `VERIFY_DATA_FLOW_DISABLED→DISABLED`, `VERIFY_RESTORE_BLOCKED→BLOCKED`

This matches AGENTS.md §6.4 exactly. No other phases, operations or states are possible.

The proposal's treatment table (`DELETE` → `REPORT_REMAINING_RECORD` / `DELETE_RECORD` / `VERIFY_RECORD_ABSENT`, `TOMBSTONE` → redact and verify redacted, `RETAIN` → verify present only, and so on) is now applied inside `policy_engine.py` rather than in P4.

3. Which steps are written

Status	FINDING	REMEDIATION	VERIFICATION
`INCOMPLETE`	One per unsatisfied record or active flow/restore. **Must** cite evidence	One per finding. `evidence_id=""`	One per assessed item, including items that were already compliant. `evidence_id=""`
`COMPLETE`	None (`build_steps()` raises if any)	None (raises if any)	Only items whose compliant state is proven at `audit_at`. **Must** cite evidence
`NOT_APPLICABLE`	—	—	— (`build_steps()` returns `[]`)

- `decide()` sets `INCOMPLETE` when there is at least one finding, and `COMPLETE` otherwise.
- For a `COMPLETE` case, a required state that no evidence proves is left out of the steps and listed in `decision["unverified"]`, so it never produces an unbacked step.
- `NOT_APPLICABLE` is decided by P1's classifier in `main.py`. `decide()` is not called and the row is written as `NOT_APPLICABLE, []`. `build_steps()` also returns `[]` for that status, as a safeguard.
- If `audit_at` is empty or the email cannot be resolved, `main.py` writes `INCOMPLETE, []` without calling P3 or P4. In the current 518-case run this happens 0 times.

4. Evidence ID rules

- **FINDING**: a non-empty `evidence_id` is required (the `file_id`, `activity_id` or `audit_id` that P3 chose). If it is empty, `ValueError` is raised.
- **REMEDIATION and VERIFICATION in an `INCOMPLETE` case**: P4 forces `evidence_id=""` even if P3 supplied one, because these steps describe future work.
- **VERIFICATION in a `COMPLETE` case**: a non-empty `evidence_id` is required. If it is empty, `ValueError` is raised.
- P4 does **not** re-check that a cited id exists or was captured at or before `audit_at`. P3 selects only evidence available at `audit_at`. `validate_output.py` checks the existence and capture time of every cited id across the whole `output.csv`.

5. Ordering and step numbering

- Phases are written in the order FINDING, then REMEDIATION, then VERIFICATION (`_PHASE_KEYS` in `step_builder.py`).
- Inside each phase, P3 sorts by the position of the operation in the AGENTS.md §6.4 table, then by `system_id`, then by `resource_id`. Within REMEDIATION, this puts `DISABLE_DATA_FLOW` and `BLOCK_RESTORE` before `DELETE`, `REDACT` and `ANONYMIZE`.
- `resource_id` **uses numeric natural ordering, not lexical ordering.** P3's `_rid_key()` in `policy_engine.py` splits the id into a prefix and a trailing number and compares the number as an integer, so `account_9` comes before `account_10`. This is existing P3 behaviour, and P3 has confirmed it. `tests/test_policy_engine.py::test_resource_ids_use_numeric_natural_sorting` covers it as a regression test. The test checks both `decide()`'s findings and the FINDING steps that `build_steps()` produces.

- P3 removes duplicate `(system_id, resource_id.lower(), operation)` entries.
- `step_no` runs consecutively from 1 across the whole case. P4 assigns it, and any input `step_no` is ignored.
- P4 keeps P3's order, including the natural `resource_id` order, and does not re-sort.
- The result is the same as the proposal's single global sort, because the operation table is already grouped by phase.

6. Ambiguities and open points, with P3's answers

1 **Interface change: CONFIRMED by P3.** `decide()` owns all policy decisions, and `build_steps()` only constructs, validates and numbers steps (see "Decisions confirmed by P3"). Section 1 of this document describes the real contract. `CONTRACTS.md` §2–3 still describes the old proposal (P4 maps resources to operations and sorts), so it should be updated to match.

2 `resource_id` **tie-break: CONFIRMED by P3 as numeric natural ordering.** The `CONTRACTS.md` proposal says only "sort by `resource_id`", which could be read as lexical order. The confirmed behaviour is numeric natural ordering through P3's `_rid_key()` (`account_9` before `account_10`; see section 5). A regression test covers it. The 32 samples contain no ordering comparison where numeric natural sorting and alphabetical sorting would produce different results: no sample case has two steps with the same operation and `system_id`. In the current 518-case output, 10 such groups have more than one `resource_id`, and none of them orders differently under the two methods. So no current row depends on this choice.

3 **Status mismatch guard: ANSWERED by P3, confirmation complete.** P3 confirmed that `decide()` returns only `COMPLETE` or `INCOMPLETE`, and that `NOT_APPLICABLE` comes from P1's classifier. The theoretical case where `main.py` replaces an unexpected status with `INCOMPLETE` while `build_steps()` still sees the original status therefore cannot occur under the confirmed contract. P4 needs no change.

4 **At least one finding per `INCOMPLETE` case: ANSWERED by P3, confirmation complete.** P3 confirmed that every `INCOMPLETE` case has at least one finding. `decide()` guarantees this by construction (status is `INCOMPLETE` only when findings exist), and `validate_output.py` checks it across `output.csv`. As agreed, `build_steps()` does not make this policy decision.

5 **OPEN TEAM QUESTION: a missing retained record is silently ignored (possible gap against AGENTS.md §6.5).** `CONTRACTS.md` §4 Q3 asks what should happen when a `RETAIN` record is missing at `audit_at`. In `_assess()` (`policy_engine.py`, the "no copy present at audit_at" branch), a `RETAIN` or `ACCOUNT_TOMBSTONE` resource with no present copy returns `None`. It produces no step and does not affect the status. AGENTS.md §6.5 says a case is `COMPLETE` only when "every required retained record remains available". No allowed operation can restore such a record. Possible options:

   6 mark the case `INCOMPLETE` with a `VERIFY_RECORD_PRESENT` step;

   7 keep the current behaviour and record the case in `notes` / `unverified`.

This is still unresolved. The team (P3 first) needs to decide. P4 has not changed this policy.

1 **The other open questions in `CONTRACTS.md` §4 are answered by the implementation, but nobody has confirmed the answers:**

   2 Q1: `system_activity.csv` is read by `activity.py`, which P3 uses.

   3 Q2: replica copies are kept separately inside a resource (`copies`). The strictest required action across the copies that are present decides the resource, and one step is emitted per `(system_id, resource_id)`. An absence is only verified when every copy that was ever seen present has a later proven absence.

   4 Q4: `rule_id` and replica details are included in `resources`.

7. Evidence that the implementation works

- `tests/test_policy_engine.py::test_step_builder_numbers_phases_and_blanks_future_evidence`, `::test_step_builder_rejects_values_outside_contract`, `::test_active_sync_and_restore_risks_and_safe_order` and `::test_resource_ids_use_numeric_natural_sorting`, plus the determinism and public-sample tests, all pass.

- The full suite gives 476 passed, 1 skipped and 592 subtests passed. Command: `cd code && python -m pytest -q tests test_identity_resolver.py test_main.py test_request_classifier.py test_utils.py`.
- `python code/main.py && python code/validate_output.py` gives **VALID** for 518 rows. The regenerated `output.csv` is byte-identical to the committed file after line-ending normalisation (SHA-256 `a215494b…97b6ef`).

8. Message to send to P3

Sent to P3. P3's answers are recorded in "Decisions confirmed by P3" and §6. Question 4 is still an open team question. The original message is kept below for reference.

> Hi P3, P4 here. I've checked the P3→P4 hand-over against the code at `cafd103`. The details are in `code/P4_CONTRACT_CONFIRMATION.md`. Could you confirm or correct these points?
>
> 1. The real contract is: `decide()` returns `deletion_status` plus ordered `findings` / `remediation` / `verification` lists of `{system_id, resource_id, operation, state, evidence_id}.build_steps()` only validates, applies the evidence-id rules and numbers the steps. Do you agree that we replace the §2–3 PROPOSAL in `CONTRACTS.md` with this?
> 2. I've documented that `resource_id` is sorted in numeric natural order by your `_rid_key()` (`account_9` before `account_10`), and I've added a regression test for it. P4 keeps that order. Please shout if lexical order was intended.
> 3. Can you confirm that `decide()` only ever returns `COMPLETE` or `INCOMPLETE`, and that an `INCOMPLETE` decision always has at least one finding?
> 4. When a `RETAIN` or tombstone record is missing at `audit_at`, `_assess()` currently drops it. It produces no step and does not change the status. AGENTS.md §6.5 says COMPLETE requires every retained record to remain available. Should such a case be INCOMPLETE (`CONTRACTS.md` §4 Q3)? This is an open team question.
>
> Once you reply, I'll update `CONTRACTS.md` and remove the PROPOSAL labels.

Appendix G: Identity and classification notes

docs/README_identity.md

Identity resolution and request classification (Person 1)

Two functions for the first pipeline stages. Both are pure Python, need no API key and give the same result every run. Import them from inside `code/`.

```
from identity_resolver import IdentityResolver
from request_classifier import classify_request
```

`classify_request(request_text) -> ClassificationResult`

Decides whether a request actually asks for deletion. Call it first: a `NOT_APPLICABLE` result means output `NOT_APPLICABLE` with `steps_json=[]` and skip the rest of the pipeline.

Input: `request_text` (str), any of English, Spanish, Portuguese, French, Japanese, Korean.

Output: `ClassificationResult` (from `models.py`):

Field	Value
`label`	`IS_DELETION`, `NOT_APPLICABLE` or `UNCERTAIN`
`confidence`	`HIGH` (clear decision) or `LOW` (only for `UNCERTAIN`)
`matched_keywords`	matched phrases, tagged `DELETION:`, `NEGATION:`, `KEEP:` or `OTHER:`
`note`	short internal note, e.g. `deletion requested`
`language`	`en`, `es`, `pt`, `fr`, `ja` or `ko`, from `detect_language()`
`signals`	the same phrases as `matched_keywords`, as `"family: phrase"`, with `OTHER` split by intent: `access`, `correction`, `opt_out`, `login` (e.g. `"opt_out: marketing"`)
`reason`	one plain-English sentence explaining the decision, quoting what matched
`is_deletion()`	`True` only for `IS_DELETION`

`language`, `signals` and `reason` only explain the decision. They are computed from the same matches and never change `label`, `confidence` or `matched_keywords`. `detect_language()` uses the script for Japanese (kana/Han) and Korean (Hangul), and typical short words and letters (ñ, ã, è ...) for English, Spanish, Portuguese and French.

Rules: `IS_DELETION` only if it asks for deletion with no negated instruction ("don't delete", "not asking for deletion", "ne supprimez pas", "삭제하지 마"). A negated description such as "was not removed" or "do not hesitate to delete" is still a deletion request. It also needs no keep-the-account phrase ("keep the account", "continue using"). A negation or keep phrase with no other deletion request gives `NOT_APPLICABLE`, and so does a non-deletion intent on its own (access, copy, correct, update, opt-out, login help). Conflicting signals, or no signal at all, give `UNCERTAIN`, so review those by hand.

Example:

```
r = classify_request("I can't access my profile and need help resetting the login. Do not close the account.")
r.label             # 'NOT_APPLICABLE'
r.matched_keywords  # ['NEGATION:Do not close', "OTHER:can't access", 'OTHER:resetting', 'OTHER:login']
r.signals           # ['negation: Do not close', "login: can't access", 'login: resetting', 'login: login']
r.language          # 'en'
r.reason            # 'Not a deletion request: the person asks for help signing in ("can't access",
                    #  "resetting"), and they say they do not want deletion ("Do not close").'
```

On the real data: case_533 to case_542 are `NOT_APPLICABLE`, the other 508 are `IS_DELETION`, and no case is `UNCERTAIN`.

`IdentityResolver(identities_path, audit_log_path).resolve(email, audit_at=None, requested_at=None, case_id="") -> IdentityBundle | None`

Maps the request email (current or historical, any letter case) to the user and gives every identifier you need to search evidence for.

Inputs:

- `identities_path`, `audit_log_path`: paths to `dataset/user_identities.csv` and `dataset/user_audit_log.csv` (loaded once, in the constructor)
- `email`: the request's `email` field
- `requested_at`: the request's `requested_at` as a `datetime`. The email resolves to whoever owned it at this time (see "For P3", section 4).
- `case_id`: only used to label warnings
- `audit_at`: a `datetime`; parse the CSV string with `utils.parse_ts()`. A naive datetime is treated as UTC. Always pass the case's `audit_at`, so that identifiers that only appeared after it are left out.

Timestamps: use `utils.parse_ts(value, context)` everywhere. It returns an aware UTC `datetime`, treats a missing timezone as UTC, returns `None` with a warning for an empty value, and raises a clear `ValueError` for text that isn't a date. An audit-log row with an empty `event_time` is kept as if it happened before `audit_at`.

Output: `IdentityBundle` (from `models.py`), or `None` if the email is unknown:

Field / method	Meaning
`user_id`	stable user reference, e.g. `user_41`
`current_email`, `current_phone`, `current_email_hash`	values held **at** `audit_at`
`current_name`	name from the snapshot
`historical_emails`, `historical_phones`, `historical_email_hashes`	earlier values up to `audit_at`
`all_emails()`, `all_phones()`, `all_email_hashes()`	current + historical, no duplicates. Use these to search evidence
`request_email`	the email exactly as given in the request

`email_hash` is SHA-256 of the lowercased email. Hashes of all historical emails are included.

Example:

```
from datetime import datetime, timezone
resolver = IdentityResolver("../dataset/user_identities.csv", "../dataset/user_audit_log.csv")
b = resolver.resolve("javier.r940897@outlook.com",
                     audit_at=datetime(2026, 10, 1, tzinfo=timezone.utc))
b.user_id            # 'user_41'
b.current_email      # 'javier.r9408@outlook.com'
b.historical_emails  # ['javier.r940897@outlook.com']
```

`resolver.resolve_all(request_rows)` does the same for a list of CSV row dicts (it reads `email` and `audit_at` from each row) and returns `(row, bundle)` pairs. All 518 requests resolve.

Tests

```
python3 -m pytest tests -v                              # whole suite (run from the repo root)

python3 tests/test_utils.py               # 13 tests
python3 tests/test_identity_resolver.py   # 66 tests
python3 tests/test_request_classifier.py  # 31 tests, 64 made-up sentences, language + reason
python3 tests/test_main.py                # 18 tests: one bad case never stops the run, P2 -> P3 contract, audi
  t_at hand-over
python3 tests/test_audit_db.py            # 22 tests: audit.db tables, save_case(), re-runs, DB errors in main.
  py
python3 tests/test_export_web_data.py     # 17 tests: JSON export and masking
```

For P3

1. The exact `IdentityBundle` structure

Defined in `code/models.py`. Every field:

Field	Type	Meaning
`user_id`	`str`	Stable user reference, e.g. `"user_41"`. Never changes, and it is the reference an `ACCOUNT_TOMBSTONE` keeps.
`current_email`	`str`	Email held **at** `audit_at`, as written in the data
`current_name`	`str`	Name from `user_identities.csv`. The audit log has no name changes, so this is never rolled back.
`current_phone`	`str`	Phone held at `audit_at`
`current_email_hash`	`str`	SHA-256 of the lowercased email, held at `audit_at`

Field	Type	Meaning
historical_emails	list[str]	Earlier emails (lowercased), from changes at or before audit_at
historical_phones	list[str]	Earlier phones, same rule
historical_email_hashes	list[str]	Earlier hashes: stored email_hash changes, plus the SHA-256 of every historical email
request_email	str	The request's email, exactly as given (not lowercased)
history	list[IdentityChange]	Every audit-log change at or before audit_at, oldest first (ties broken by audit_id)

Methods: `all_emails() -> list[str]`, `all_phones() -> list[str]`, `all_email_hashes() -> list[str]`. Each returns the current value first, then the historical ones, with no duplicates (`all_emails()` lowercases).

`IdentityChange` (also in `models.py`) is a frozen dataclass:

Field	Type	Meaning
audit_id	str	Row ID in user_audit_log.csv, e.g. "audit_293"
field_name	str	"email", "phone" or "email_hash"
old_value	str	Value before the change
new_value	str	Value after the change
event_time	str	ISO-8601 UTC string exactly as in the CSV, e.g. "2026-05-28T19:48:13Z"

2. How current and historical identifiers are represented

- **Current** identifiers are single strings: `current_email`, `current_phone`, `current_email_hash`.
- **Historical** identifiers are plain lists of the `old_value`s of past changes: `historical_emails`, `historical_phones`, `historical_email_hashes`.
- To search evidence, use `all_emails()`, `all_phones()` and `all_email_hashes()`, which combine both. Compare emails in lowercase. Phones are kept exactly as written (e.g. `"+34 695 783 595"`), so normalise them yourself if evidence formats phones differently.

3. How historical changes link to `audit_id`, `old_value`, `new_value` and `event_time`

Each entry in `bundle.history` is one row of `user_audit_log.csv`, with all five fields. The lists in section 2 hold only the `old_value`s. To find which change produced an old identifier, or to cite it as `evidence_id`, search `history`:

```
change = next(c for c in bundle.history
              if c.field_name == "email" and c.old_value.lower() == some_email)
change.audit_id    # 'audit_293'
change.event_time  # '2026-05-28T19:48:13Z'  (stopped being current at this time)
```

Hash changes are separate rows (`field_name="email_hash"`). In this dataset they always have the same `event_time` as the email change they belong to. Changes with an empty `old_value` stay in `history` but don't add anything to the historical lists.

4. How identifiers are resolved as of `audit_at`

`resolve(email, audit_at, requested_at, case_id)` does this:

1. **Find who owned the email when the request was sent.** From `user_identities.csv` and the email changes in `user_audit_log.csv`, it knows the periods each user owned each email (a change's `event_time` is when the new value starts). The lowercased request email resolves to the user who owned it at `requested_at`, or at `audit_at` if `requested_at` is missing, or the current owner if neither is given. So if an email moved from user A to user B, a request sent before the move resolves to A.
2. If several users could have owned it at that time (e.g. two users list the same old email with unknown start dates), it prints a warning with the `case_id` and picks the user who gave the email up first, assuming the email passed from one user to the next.
3. If nobody owned it at that exact time, it warns and picks the nearest owner in time (the last owner before, otherwise the first after).
4. In today's dataset no email ever belonged to two users, so this never triggers and every case resolves exactly as it did before.

5 **Roll the snapshot back to** `audit_at`**.** `user_identities.csv` shows the latest state. Every change with `event_time > audit_at` is undone, newest first: email, phone and hash are set back to that change's `old_value`. So `current_*` shows what the user actually had at `audit_at`, and a value that only appeared later is not in the bundle at all. If an email is rolled back with no matching hash row, the hash is recalculated as SHA-256.

6 **Keep only past history.** `history` and the `historical_*` lists contain only changes with `event_time <= audit_at`. A change exactly at `audit_at` is included.

If you call it without `audit_at`, there is no roll-back and the full history is used. Always pass `audit_at` for a case.

In today's dataset no case has a change after its `audit_at`, so the roll-back never triggers. It is there for hidden or edge-case data.

5. The function to call for a deletion request

`main.py` already calls this and passes the bundle to `find_records()` and then `decide()`, so you receive it as the `identity` argument. To call it yourself:

```
from identity_resolver import IdentityResolver
from utils import parse_ts

resolver = IdentityResolver("dataset/user_identities.csv",   # build once
                            "dataset/user_audit_log.csv")

audit_at = parse_ts(case["audit_at"], context=f"{case['case_id']} audit_at")
requested_at = parse_ts(case["requested_at"], context=f"{case['case_id']} requested_at")
identity = resolver.resolve(case["email"], audit_at=audit_at,
                            requested_at=requested_at, case_id=case["case_id"])
if identity is None:
    ...  # unknown email (doesn't happen in the current data)
```

Use `parse_ts()` for every timestamp (`audit_at`, `event_time`, evidence capture times) so all comparisons are between aware UTC datetimes. If you raise an exception, `main.py` writes that case as `INCOMPLETE, []` and lists it at the end of the run, so check the summary.

6. Real sample: case_43

Input row from `deletion_requests.csv`:

```
case_id=case_43  email=javier.r940897@outlook.com
requested_at=2026-09-15T11:00:00Z  audit_at=2026-09-26T11:00:00Z
```

The request uses an **old** email. The actual printed result of `resolver.resolve(case["email"], audit_at=audit_at)`:

```
IdentityBundle(user_id='user_41',
               current_email='javier.r9408@outlook.com',
               current_name='Javier Ríos',
               current_phone='+34 695 783 595',
               current_email_hash='5e3b9b928e5c07b78012b06e451a02a26789e1639766588b7b706b8bbecf89bd',
               historical_emails=['javier.r940897@outlook.com'],
               historical_phones=[],
               historical_email_hashes=['2e0aabf7d2e6c922c1d59fe49b69d3d9874d2f83ccaf826538f48e84e0ef9515'],
               request_email='javier.r940897@outlook.com',
               history=[IdentityChange(audit_id='audit_293',
                                       field_name='email',
                                       old_value='javier.r940897@outlook.com',
                                       new_value='javier.r9408@outlook.com',
                                       event_time='2026-05-28T19:48:13Z'),
                        IdentityChange(audit_id='audit_294',
                                       field_name='email_hash',
                                       old_value='2e0aabf7d2e6c922c1d59fe49b69d3d9874d2f83ccaf826538f48e84e0ef95
  15',
                                       new_value='5e3b9b928e5c07b78012b06e451a02a26789e1639766588b7b706b8bbecf89
  bd',
                                       event_time='2026-05-28T19:48:13Z')])

all_emails()       -> ['javier.r9408@outlook.com', 'javier.r940897@outlook.com']
all_phones()       -> ['+34 695 783 595']
all_email_hashes() -> ['5e3b9b92…bf89bd', '2e0aabf7…ef9515']
```

Both emails (and both hashes) must be searched in evidence for this case. `audit_293` and `audit_294` (2026-05-28) are before `audit_at` (2026-09-26), so they are included.

For P2 and P3 – adapter

`IdentityBundle.as_subject_kwargs()` turns the bundle into the keyword arguments of P2's `SubjectIdentifiers` (`code/evidence/models.py` on P2's branch). This branch does not import P2's code: the method returns a plain `dict`.

Usage for P2 (one line)

```
subject = SubjectIdentifiers(**identity.as_subject_kwargs())
```

`identity` is the `IdentityBundle` that `main.py` passes to `find_records(identity, case)`.

Mapping

`SubjectIdentifiers` field	Type	Taken from `IdentityBundle`
`user_id`	`Optional[str]`	`user_id`
`emails`	`frozenset[str]`	`all_emails()`: current + historical, lowercased
`phones`	`frozenset[str]`	`all_phones()`: current + historical, exactly as written
`email_hashes`	`frozenset[str]`	`all_email_hashes()`: current + historical SHA-256
`names`	`frozenset[str]`	`{current_name}`, or empty if there is no name
`resource_ids`	`frozenset[str]`	always empty. P2 fills these from evidence

Every value is "as of `audit_at`": identifiers that only appeared after `audit_at` are not included. Empty strings are dropped, so a missing phone or name never becomes a search term that matches blank fields in evidence.

Note for P3: dates and evidence IDs

`SubjectIdentifiers` has no `valid_from` / `valid_to`, so it can't tell you **when** an identifier was in use or **which audit row** proves it. For that, use `identity.history` (a list of `IdentityChange`, oldest first, only changes at or before `audit_at`):

Field	Use it for
`audit_id`	the `evidence_id` to cite for an identity change
`old_value`	the identifier used **until** `event_time`
`new_value`	the identifier used **from** `event_time`
`event_time`	the date of the change (ISO-8601 UTC string; parse with `utils.parse_ts()`)

For example, an old email in `subject.emails` was in use until the `event_time` of the `history` entry whose `old_value` is that email.

Real example: case_43

```
identity = resolver.resolve(case["email"], audit_at=audit_at,
                            requested_at=requested_at, case_id="case_43")
identity.as_subject_kwargs()
```

Actual output (sets shown sorted):

```
{'user_id':      'user_41',
 'emails':       frozenset({'javier.r940897@outlook.com',      # old, used in the request
                            'javier.r9408@outlook.com'}),      # current
 'phones':       frozenset({'+34 695 783 595'}),
 'email_hashes': frozenset({'2e0aabf7d2e6c922c1d59fe49b69d3d9874d2f83ccaf826538f48e84e0ef9515',   # old
                            '5e3b9b928e5c07b78012b06e451a02a26789e1639766588b7b706b8bbecf89bd'}), # current
 'names':        frozenset({'Javier Ríos'}),
 'resource_ids': frozenset()}
```

Both the old and the current email and hash are included. The dates and evidence IDs for the change are in `identity.history`: `audit_293` (email) and `audit_294` (email_hash), both at `2026-05-28T19:48:13Z`.

Appendix H: Evidence scanner README

`code/evidence/README.md`

Evidence Discovery module (`code/evidence/`)

Discovers and reads every file listed in `dataset/evidence_files.csv` and exposes normalised observations to the auditor. **It makes no policy decisions**: no `deletion_status`, no `steps_json`, and it never drops retained, tombstone or replica records.

Setup and run

```
python -m venv .venv                        # at the repository root
.venv/Scripts/pip install -r code/requirements.txt     # Windows
# .venv/bin/pip install -r code/requirements.txt       # macOS / Linux

python code/evidence/__main__.py --per-file             # full-corpus scan report
cd code && python -m pytest                             # tests
RUN_OCR_TESTS=1 python -m pytest -m ocr                 # also run the live OCR engine
python code/evidence/ocr_cache_build.py [--force]       # rebuild the OCR cache
```

API (used from `code/main.py`)

```
from evidence import EvidenceScanner, SubjectIdentifiers

scanner = EvidenceScanner(dataset_dir)        # ocr_mode="auto" | "cache_only" | "force"
scanner.load()                                # parse all 94 files once

ids = SubjectIdentifiers(user_id="user_05", emails=frozenset({...}), phones=..., email_hashes=...,
                         names=..., resource_ids=frozenset({"account_001"}))   # resource ids e.g. from system_a
  ctivity
audit_at = datetime(2026, 9, 3, 11, 30, tzinfo=timezone.utc)   # must be timezone-aware
hits   = scanner.find_records(ids, audit_at)          # list[ObservedRecord]: every observation, claims included
latest = scanner.latest_by_resource(ids, audit_at)    # {(system_id, resource_id, replica_or_table): ObservedRec
  ord}
                                                      #   authoritative only; include_claims=True to let claims
  compete
proofs = scanner.absence_proofs("account_service", "account_001", audit_at)   # list[AbsenceProof], COMPLETE onl
  y
                                                      #   include_unverified=True adds INFERRED / INCOMPLETE can
  didates
scanner.records_for_resource("analytics_9406", audit_at)
scanner.files(); scanner.get_file("evidence_54"); scanner.issues; scanner.summary()
```

`find_for_subject(ids, audit_at)` is kept as a backward-compatible alias of `find_records` (identical results).

```
refs = scanner.referenced_resources(ids, audit_at)    # list[ResourceReference]: ids named inside the subject's
                                                      #   confirmed, authoritative, in-window observations
for r in refs:                                        #   e.g. account_006 via evidence_57 source_resource_id
    scanner.absence_proofs(r.system_id, r.resource_id, audit_at)
```

- **Identity assessment** (on every record returned by `find_records`): `match_strength` is `STRONG` (email / phone / email_hash / user_id / supplied resource id), `LINKED` (followed from a confirmed observation; `linked_via` gives `evidence_id@locator`) or `WEAK` (exact full name only; a name never establishes ownership). `identity_conflict` + `conflict_details` flag records whose stored `user_*` reference disagrees with the subject's `user_id`, or that carry the subject's `user_id` with a different owner email / phone / hash. `rec.is_confirmed` = STRONG or LINKED and no conflict. WEAK and conflicting records are still returned, but they never seed link expansion or `referenced_resources`, and `latest_by_resource` ignores them unless `include_unconfirmed=True`. Claim-only records may point at their own resource id but their cross-references are never followed.
- **File-name links** use only real file-name fields (`file_name`, `file`, screenshot file names). Archive paths and object keys (`segment-31.tar.zst`, `customer_data.zip`) are shared storage locations and are never used to link records.
- `audit_at` **validation**: a timezone-aware `datetime`, or an ISO-8601 string with an explicit offset (`Z` / `+HH:MM`). Naive datetimes and offset-less strings raise `ValueError`; other types raise `TypeError`.
- **Cutoffs**: a record is returned only if the file's manifest `captured_at` **and** the record's own `observed_at` are <= `audit_at` (inclusive).

- `replica_or_table` is `None` for the primary / system-of-record copy and the replica name otherwise (`customer_profiles_failover_replica`, `customer_data_read_replica`, `support_replica`, ...).
- `presence`: `PRESENT`, `ABSENT` (404 / rows=0 / NOT_FOUND / zero hits) or `UNKNOWN`.
- `claim_only=True` marks second-hand lookup ledgers (handover TXT, `record_lookup` logs, export-manifest / connector / security-audit ledgers, `operational_record_history`). Snapshots, SQL rows, completed read-backs, PII scans, PDFs and screenshots are `claim_only=False`.
- `pii_fields` holds only direct identifiers stored in the record (non-null). Query keys such as `lookup_key` link a record to a subject but are never reported as exposed PII. A tombstone with null name/email/phone/hash is `PRESENT` with `pii_fields == {}` and keeps its `subject_reference`.
- **Linking**: identifiers match exactly after normalisation (emails case-insensitive, phones digits only, names case/accent-insensitive full-name equality, `user_*` exact). Direct matches are then expanded through resource ids (and attachment file names) so de-identified later copies are found.
- **Absence**: `SnapshotScope`s record which collections a snapshot fully enumerated (JSON bundles, collection dumps, SQL tables, inventory `snapshot_scope`). `absence_proofs` returns the in-window scopes of the same system and data category that do not contain the resource. Each `AbsenceProof` carries `scope_source`, `completeness` (`COMPLETE` / `INFERRED` / `INCOMPLETE`), `scope_record_count`, `limitations` (which copy it covers, inference caveats), `errors` (parse issues of the file) and `contradicted_by` (later authoritative PRESENT observations of the same copy). Only `COMPLETE` proofs are returned by default: coverage inferred from sibling SQL tables, and files with parse issues, are never treated as proof. `proof.is_valid_proof` = COMPLETE and not contradicted; `proof.verified_at` is the snapshot time.

Layout

File	Role
`models.py`	`EvidenceFile`, `SubjectIdentifiers`, `ObservedRecord`, `SnapshotScope`, `AbsenceProof`, `ParseIssue`
`manifest.py`	loads and validates the manifest (missing / unlisted / duplicate / unsupported files)
`normalize.py`	identifier normalisation, free-text extraction, resource-prefix and collection vocabularies
`parsers/json_records.py`	snapshot bundles, service exports, lookup ledgers, collection dumps
`parsers/sql_snapshots.py`	executes each dump in in-memory SQLite; blob, inventory and typed tables
`parsers/logs.py`	6 line syntaxes / 9 file families, trace-id correlation of `*_started` and `*_completed`
`parsers/pdf_documents.py`	pypdf text; register rows, resource mentions, single-subject documents
`parsers/images.py`	RapidOCR + SHA-256-keyed cache in `ocr_cache/`, row-aware identifier linking
`parsers/text_handover.py`	pipe-delimited handover checklist
`index.py`, `scanner.py`	inverted index and the public API
`integration.py`	P1 identity bundle -> findings dictionary for P3 (`find_evidence`, `get_scanner`)

Integration with P1 and P3 (`integration.py`)

```
from evidence.integration import find_evidence, get_scanner

findings = find_evidence(identity, audit_at)   # identity: P1 IdentityBundle; audit_at: tz-aware datetime
# optional: scanner=..., or dataset_dir=... / ocr_mode=... for get_scanner()

from evidence.integration import find_case_evidence
findings = find_case_evidence(identity, case)  # case: a deletion_requests.csv row (dict)
```

`find_case_evidence(identity, case)` returns exactly `find_evidence(identity, case_audit_at(case))`. `case_audit_at(case)` accepts `case["audit_at"]` as a timezone-aware `datetime` or an ISO-8601 string with an explicit offset (`Z`, `+HH:MM`, `-HHMM`), as stored in the CSV. Missing/empty values, naive datetimes and offset-less strings raise `ValueError`; other types, or a `case` that is not a mapping, raise `TypeError`.

- Identifiers come from P1's adapter exactly: `SubjectIdentifiers(**identity.as_subject_kwargs())`. Nothing from P1 is imported; only the attributes documented in `docs/README_identity.md` are read (`as_subject_kwargs()`, `user_id`, `current_email`, `current_phone`, `current_email_hash`, `current_name`, `history[*].audit_id/field_name/old_value/new_value/event_time`).
- `get_scanner(dataset_dir=None, *, ocr_mode="auto")` loads the corpus once per process for each (resolved dataset directory, OCR mode); `find_evidence` uses it unless `scanner=` is passed.

- One validated cutoff is used for every scanner call. `audit_at` missing or naive -> `ValueError`; not a `datetime` -> `TypeError`. A `history` row dated after `audit_at` -> `ValueError` (the bundle was resolved for another cutoff). An identifier in `as_subject_kwargs()` that neither the snapshot fields nor `history` explain -> `ValueError` (nothing is guessed).
- The scanner APIs (`find_records`, `latest_by_resource`, `referenced_resources`, `absence_proofs`) are called unchanged with their safe defaults.

Returned dictionary (exactly five keys)

Key	Type	Semantics
`records`	`list[ObservedRecord]`	`scanner.find_records(identifiers, audit_at)`: every in-window observation, including claim-only (`claim_only=True`), name-only (`match_strength="WEAK"`) and conflicting (`identity_conflict=True`) ones, with `identifiers_matched` and `linked_via` provenance. Nothing is collapsed.
`latest_by_resource`	`dict[tuple[str, str, Optional[str]], ObservedRecord]`	Key `(system_id, resource_id, replica_or_table)`, `replica_or_table=None` for the primary copy. Latest **authoritative, confirmed** observation per copy (`scanner.latest_by_resource` defaults): no claims, no WEAK, no conflicts.
`referenced_resources`	`list[ResourceReference]`	Resource ids named inside confirmed, non-claim, in-window observations (e.g. `source_resource_id`), with source evidence id, locator and field. Pointers only; `has_observations` says whether the resource has any in-window observation.
`absence_proofs`	`dict[tuple[str, str], list[AbsenceProof]]`	Key `(system_id, resource_id)`, deduplicated case-insensitively. One key per referenced resource with a known `system_id` and per resource of a confirmed, non-claim observation. Values hold only valid proofs (`completeness == "COMPLETE"` and not `contradicted_by`); an empty list means the scanner could establish no valid proof. Claims never create a key.
`identifier_provenance`	`dict[tuple[str, str], list[dict]]`	Key `(kind, normalised_value)` for every searched identifier, normalised as the scanner matches (so `f"{kind}:{value}"` equals an `identifiers_matched` label). `kind` is `user_id`, `email`, `phone`, `email_hash` or `name`. Each entry: `{"source", "audit_id", "event_time", "basis"}` (see below).

Provenance entries:

Field	Type	Meaning	
`source`	`"CURRENT" \`	`"HISTORICAL"`	HISTORICAL = an `old_value` in `identity.history`; CURRENT = the bundle's current email/phone/hash, `user_id` or `current_name`. A value that was both gets one entry each.
`audit_id`	`str \`	`None`	HISTORICAL: the history row whose `old_value` it is. CURRENT: the row whose `new_value` made it current, else `None`. `None` for computed hashes. Never invented.
`event_time`	`datetime \`	`None`	Aware UTC. HISTORICAL: when it **stopped** being current (P1: "used until `event_time`"). CURRENT: when it **became** current. `None` if P1's `event_time` is empty or there is no row.
`basis`	`str`	`history.old_value`, `history.new_value`, `snapshot`, or `computed_sha256_of_historical_email` (P1 adds the SHA-256 of each historical email; such a hash may have no audit row).	

Historical identifiers are included because their change is at or before `audit_at`; evidence is **not** filtered by the identifier's validity window. Old identifiers that still appear after they were changed are exactly the ghost data the auditor must find (all 214 historical matches in the corpus are of this kind).

Appendix I: AGENTS.md: rules for coding agents

AGENTS.md

AGENTS.md

HackerRank Orchestrate (October 2026) — Ghost Data Still Exists

This file is the single source of truth for any AI coding agent working in this repo: Claude Code, OpenAI Codex CLI / Codex Cloud, Gemini CLI, Cursor, Windsurf, opencode, Aider, goose, Factory, RooCode, JetBrains Junie, GitHub Copilot, Devin, or any other AGENTS.md-aware tool.

Read this file in full before taking any action. Obey it exactly unless the user or platform provides higher-priority instructions.

0. TLDR For The Agent

On every session start, do this in order:

1. Read this file completely.
2. Check the log file path in §2.
3. Append a `SESSION START` entry using §5.1.
4. For every user turn, append a summary entry using §5.2.
5. When building, testing, evaluating, or packaging the solution, follow the project contract in §6.

Do not skip logging or rewrite old log entries. Sub-agents and worktrees use the same log file.

1. What This Repo Is

This is a starter repo for the **HackerRank Orchestrate** challenge: **Ghost Data Still Exists**.

Participants must build an AI-powered deletion-audit agent. For every request in `dataset/deletion_requests.csv`, the agent first determines whether the person actually requested deletion. For an authorized deletion, it reconstructs the person's current and historical identifiers, searches global activity and evidence, applies the supplied deletion policy, and decides whether the deletion is complete.

The dataset models a commerce platform where a workflow can report **Complete** while personal data remains in databases, replicas, logs, support attachments, exports, search indexes, analytics records, archives, or backups. Active synchronization and restore processes can also recreate data that was deleted from its original source. Reported statuses and activity results are claims; the agent must establish the state at each case's `audit_at` time from the supplied evidence.

When deletion is incomplete, the agent reports the remaining records and recreation risks, produces a safe ordered remediation plan, and states the verification checks that must pass afterward. When deletion is complete, it returns the evidence-backed verification checks that prove the required state. It must preserve records required by policy and correctly handle account tombstones whose stable internal user reference remains while prohibited direct identifiers are removed.

The final submission must produce `output.csv` with:

```
case_id,deletion_status,steps_json
```

Read `problem_statement.md` for the full participant-facing specification.

2. Log File — Location And Lifecycle

The log file is named `log.txt` and lives in the same directory as this `AGENTS.md` file — the repository root.

Platform	Path
macOS / Linux	`<directory containing AGENTS.md>/log.txt`
Windows	`<directory containing AGENTS.md>\log.txt`

Resolve the path relative to this file. Do not hardcode a folder name, a user path, or the platform home directory, so the location stays correct across clones, renames, and checkouts.

Rules:

- Create the file if missing.
- Append only. Do not rewrite, reorder, or delete prior entries.
- One shared log is used per checkout. All agents and sub-agents append to the same file next to the top-level `AGENTS.md`, never a private copy.
- Never log secrets. Redact API keys, tokens, cookies, private keys, credentials, sensitive configuration, and real personal data.

3. Session Start

At the beginning of each agent session:

1 Append a short `SESSION START` entry using §5.1.

2 Greet the user with this brief introduction:

```
 Welcome to HackerRank Orchestrate. Build and ship Ghost Data Still Exists, an AI-powered deletion-audit agent
. Let's get started.
```

1 The October challenge end time is not configured in this repository. Do not invent a deadline or reuse the September deadline. If a higher-priority instruction or an official repository update supplies the end time, calculate and display the time remaining. If fewer than 2 hours remain, remind the user to submit soon. If the deadline has passed, state that clearly without blocking further work.

2 Proceed with the user's request without requiring an acknowledgement or confirmation phrase.

4. Challenge Rules

1 Participants may use any programming language, model, framework, IDE, AI assistant, retrieval approach, or tool, subject to the official event rules.

2 The system must conform to the project contract in §6 so it can be evaluated.

3 The solution must use only participant-facing data under `dataset/`. Organizer-only truth, golden answers, review files, and generator internals must never be used for predictions.

4 Never commit or submit secrets. Read credentials from environment variables and use an uncommitted `.env` file when needed.

5 Log every conversation turn to the file described in §2.

6 Follow the submission-link rule below.

4.1 Challenge And Submission Link

The October challenge and submission URL is:

https://www.hackerrank.com/contests/hackerrank-orchestrate-october26/challenges/ghost-data-still-exists

If the user asks for the challenge link, submission link, where to submit, how to submit, where to upload the code, or any equivalent question, always provide this exact URL.

Requirements:

- Include the full clickable URL every time such a question is asked.
- Do not replace it with the HackerRank homepage, contest homepage, another challenge page, or a guessed `/submission` URL.
- Do not merely describe where to navigate; provide the URL directly.
- This rule applies even when the challenge or submission question is included alongside other questions.

5. Log Format

5.1 Session Start Entry

```
## [ISO-8601 TIMESTAMP] SESSION START

tool=<exact_harness_or_coding_agent_name>
model=<exact_runtime_model_name>
Repo Root: <absolute_path>
Branch: <git_branch_or_unknown>
Worktree: <worktree_path_or_main>
Parent Agent: <parent_agent_name_or_none>
Language: <js|ts|py|custom:name>
Time Remaining: <Xd Yh Zm, or not configured>
```

5.2 Per-Turn Entry

Append after every user message you respond to:

```
## [ISO-8601 TIMESTAMP] <short title, max 80 chars>

User Prompt (verbatim, secrets redacted):
<exact user message, with secrets replaced by [REDACTED]>

Agent Response Summary:
<2-5 sentences: what was done, why, and any important decision>

Actions:
* <file edited / command run / tool invoked>

Context:
tool=<exact_harness_or_coding_agent_name>
model=<exact_runtime_model_name>
branch=<git_branch_or_unknown>
repo_root=<absolute_path>
worktree=<worktree_path_or_main>
parent_agent=<parent_name_or_none>
```

Mandatory tool-name rule: Every `SESSION START` and per-turn log entry must contain one non-empty `tool=` line with the exact name of the coding harness or agent writing the entry. Replace the template value before writing the log. The entry is invalid if `tool=` is missing, blank, still contains a placeholder, uses a generic label such as `AI`, contains only a model name, or names a different harness. Before responding, verify the value against the harness identity provided by the current runtime and re-read the appended entry to confirm it matches. Never guess the tool name. Correct any mismatch before responding to the user.

Mandatory model-name rule: Every `SESSION START` and per-turn log entry must contain one non-empty `model=` line with the exact model identifier used for that turn. Always make sure that model name is correct & not made up. `model=unknown`, a model family, a generic label, an inferred name, and a template placeholder are invalid. Obtain the exact identifier from the runtime, session metadata, harness configuration, or another authoritative environment source before writing the entry. Never guess the model name. The model name does not replace the `tool=` value because the coding harness and the model are separate fields.

Required harness-and-model check: Both `tool=` and `model=` must always be present in every `SESSION START` and per-turn entry. An entry is invalid if either field is missing, blank, unknown, generic, inferred, or still contains a template placeholder. Before completing a response, re-read the newly appended log entry and confirm that it contains exactly one populated `tool=` line and exactly one populated `model=` line with authoritative values. If either is absent or invalid, determine the correct value and correct the entry before responding.

5.3 Sub-Agent And Worktree Rules

- Sub-agents must log their own entries using the same file.
- Set `parent_agent=` to the parent agent's name.
- Worktrees use the same shared log file, not a per-worktree copy.

5.4 What Not To Log

- API keys, tokens, session cookies, OAuth codes, credentials, or private keys.
- Sensitive configuration or real personal data.
- Full contents of large files or binary blobs. Reference them by path instead.

6. Project Contract

6.1 Dataset Contract

Participant-facing files are inside `dataset/`.

```
dataset/
├── deletion_requests.csv
├── sample_deletion_requests.csv
├── sample_output.csv
├── user_identities.csv
├── user_audit_log.csv
├── system_activity.csv
├── deletion_rules.csv
├── evidence_files.csv
├── output.csv
└── evidence/
    ├── databases/
    ├── documents/
    ├── images/
    ├── logs/
    ├── records/
```

- `deletion_requests.csv` contains 518 evaluation requests. Produce exactly one output row for every `case_id` in this file.
- `sample_deletion_requests.csv` contains 32 public requests. `sample_output.csv` contains their correct outputs and demonstrates the required decision and serialization format.
- `user_identities.csv` contains the current identity snapshot for each user, including the stable `user_id`, name, email, phone, email hash, and snapshot update time.
- `user_audit_log.csv` records historical changes to identity fields. Use it with the current snapshot to recover identifiers that may still appear in older systems and evidence.
- `system_activity.csv` is a global stream of deletion, synchronization, export, backup, restore, indexing, and audit events. Its rows may describe unrelated people. Determine resource state from the available evidence rather than trusting workflow or operation results alone.
- `deletion_rules.csv` defines the required action for each data category and condition. Actions include deleting the whole record, redacting prohibited identifiers, anonymizing the record, retaining it, or preserving an account tombstone with only the permitted stable reference.
- `evidence_files.csv` is the manifest for evidence under `dataset/evidence/`. It maps each `file_id` to its path, source origin, and capture time. The manifest does not identify which case or user a file concerns; the solution must connect evidence through its contents and identifiers.
- `output.csv` is the blank prediction template. The final completed `output.csv` must be written at the repository root for submission.
- Evidence may be provided as PDF documents, application screenshots, logs, JSON, plain-text records, and SQLite-compatible SQL snapshot exports. SQL files are readable text and may be executed with SQLite.

Only `deletion_requests.csv` requires predictions. Other participant-facing files provide identity, policy, activity, and evidence context. Global data can contain decoys and unrelated records.

Organizer-only files live outside `dataset/` and must never be used for predictions, prompt examples, hardcoded labels, or evaluation-time shortcuts.

6.2 Input And Time Contract

Each request has these fields:

```
case_id,email,requested_at,reported_status,reported_at,audit_at,request_text
```

- `case_id` uniquely identifies the request and its output row.
- `email` is the identity supplied with the request. It may be a current or historical email address.
- `requested_at` is when the request was submitted.
- `reported_status` is the deletion workflow's claim: `COMPLETE`, `PARTIALLY_COMPLETE`, or `NOT_STARTED`.
- `reported_at` is when that claim was reported.
- `audit_at` is the cutoff for the required answer.

- `request_text` contains the person's request and may be written in different languages. Determine whether it actually authorizes deletion.

Use only information available at or before `audit_at` when deciding the case. Evidence created later must not change the answer for that case.

6.3 Required Output

The solution must write `output.csv` with these exact columns, in this order:

```
case_id,deletion_status,steps_json
```

Requirements:

- Produce exactly 518 data rows: one for every `case_id` in `dataset/deletion_requests.csv`.
- Sort rows by `case_id`.
- Set `deletion_status` to `COMPLETE`, `INCOMPLETE`, or `NOT_APPLICABLE`.
- Use `NOT_APPLICABLE` with `steps_json=[]` when the request does not authorize deletion.
- Encode `steps_json` as a valid JSON array. The grader ignores JSON object-key order and whitespace but preserves array order.
- Number steps consecutively from `1`.
- Order phases as `FINDING`, then `REMEDIATION`, then `VERIFICATION`.

Each step contains exactly these fields:

```
{
  "step_no": 1,
  "phase": "FINDING",
  "system_id": "account_service",
  "resource_id": "account_005",
  "operation": "REPORT_REMAINING_RECORD",
  "state": "PRESENT",
  "evidence_id": "evidence_01"
}
```

For a `COMPLETE` case, include only evidence-backed verification steps proving the state at `audit_at`. For an `INCOMPLETE` case, include the findings, the complete ordered remediation plan, and the checks that must pass after remediation. Findings cite participant-visible evidence. Proposed remediation and future verification steps use an empty `evidence_id`.

6.4 Allowed Step Operations

Findings:

Operation	State	Meaning
`REPORT_REMAINING_RECORD`	`PRESENT`	A record that should have been deleted still exists.
`REPORT_EXPOSED_PII`	`PRESENT`	A record may remain, but identifiers that policy requires to be redacted or anonymized are still exposed.
`REPORT_ACTIVE_DATA_FLOW`	`ACTIVE`	An active process can copy or recreate the user's data.
`REPORT_RESTORE_RISK`	`ACTIVE`	A backup or restore path can bring deleted data back.

Remediation:

Operation	State	Meaning
`DISABLE_DATA_FLOW`	`DISABLED`	Stop a process that can copy or recreate the data.
`BLOCK_RESTORE`	`BLOCKED`	Prevent the affected backup from restoring the user's data.
`DELETE_RECORD`	`ABSENT`	Remove the entire record.
`REDACT_RECORD`	`REDACTED`	Keep the record but remove the identifying fields required by policy.
`ANONYMIZE_RECORD`	`ANONYMIZED`	Keep the record after irreversibly unlinking it from the user.

Verification:

Operation	State	Meaning
`VERIFY_RECORD_ABSENT`	`ABSENT`	Confirm that a record no longer exists.
`VERIFY_RECORD_PRESENT`	`PRESENT`	Confirm that a policy-required retained record still exists.

Operation	State	Meaning
VERIFY_RECORD_REDACTED	REDACTED	Confirm that required identifying fields were removed.
VERIFY_RECORD_ANONYMIZED	ANONYMIZED	Confirm that the record can no longer be linked to the user.
VERIFY_DATA_FLOW_DISABLED	DISABLED	Confirm that the relevant data flow is inactive.
VERIFY_RESTORE_BLOCKED	BLOCKED	Confirm that the affected data cannot be restored.

Do not invent additional phases, operations, or states.

6.5 Deletion Decision Rules

- First determine whether `request_text` authorizes deletion. A request for access, correction, export, marketing opt-out, or another non-deletion action is `NOT_APPLICABLE`, even if other fields resemble a deletion case.
- Resolve both current and historical identifiers for the subject. The request email may match either form.
- Reconstruct the true state at `audit_at`. Ignore later evidence and do not accept `reported_status` or `reported_result` as proof.
- Apply `deletion_rules.csv` to every relevant record. Do not delete data that policy requires to be retained, redacted, anonymized, or preserved as a tombstone.
- For an `ACCOUNT_TOMBSTONE`, preserve the stable subject reference and remove the name, email, phone, email hash, and any other direct identifiers required by the applicable `REDACT` rule.
- A deletion is `COMPLETE` only when every relevant copy has the required treatment, every required retained record remains available, and no active sync, import, or restore path can recreate prohibited data.
- For an incomplete case, report every relevant policy violation and recreation risk supported by participant-visible evidence.
- Within remediation, disable data flows and block unsafe restores before changing dependent records. Process downstream copies before their source records when the dependency requires it.
- Verification must cover the required final state for each remediated record or process.

6.6 Evidence Rules

- Treat activity logs, workflow statuses, and system messages as claims unless the available evidence establishes the resulting state.
- Use `evidence_files.csv` to locate files, then inspect their contents. Do not infer case membership from the manifest alone.
- Match evidence through current identifiers, historical identifiers, hashes, resource IDs, system IDs, timestamps, and other grounded links.
- Images and PDFs are ordinary business artifacts such as invoices, support records, handover documents, and internal product screens. Extract the facts they contain; do not expect an explicit deletion verdict.
- SQL evidence can contain primary, regional, reporting, or failover replicas. Compare the relevant tables and snapshot time when investigating replication drift.
- A finding must use the participant-visible `file_id`, `activity_id`, or `audit_id` that proves it.
- A verification step for a `COMPLETE` case must cite the evidence that proves the current compliant state.
- Remediation steps and post-remediation verification steps for an `INCOMPLETE` case describe future work and therefore use `evidence_id` as an empty string.
- Never cite organizer-only truth or an evidence item captured after the case's `audit_at`.

6.7 Constraints That Make The Submission Evaluable

- Be runnable from the terminal.
- Read the provided files from `dataset/`.
- Write the completed `output.csv` at the repository root.
- Do not use organizer-only files, hardcoded golden labels, or case-family shortcuts.
- Keep behavior deterministic where possible, including ordering and JSON serialization.

- Read secrets from environment variables only.
- Include clear setup and run instructions in the submitted `README.md`.
- Ensure a clean run can process all 518 cases and produce exactly one valid row per case.

6.8 Evaluation And Token Usage

The submitted `code.zip` must include:

```
evaluation/evaluation_report.md
evaluation/usage_report.md
```

`evaluation/evaluation_report.md` must contain:

- case-level deletion-status accuracy;
- step-level and field-level accuracy;
- sequence accuracy;
- at least three additional evaluation cases created by the participant; and
- failures found and improvements made.

`evaluation/usage_report.md` must summarize the final full-dataset run that produced `output.csv`, including:

- model providers and model names;
- model-call counts;
- input and output tokens;
- total and average tokens per request;
- estimated total and per-request cost; and
- per-model and overall totals when multiple models are used.

Write the required evaluation metrics, custom cases, and usage details directly in these Markdown files. Never include API keys, credentials, or sensitive configuration values.

6.9 Submission Artifacts

The participant submission contains:

Item	Description
`code.zip`	Runnable solution, prompts, configuration, `README.md`, and the `evaluation/` reports
`output.csv`	One answer row for every evaluation request
`chat_transcript`	The root `log.txt` file generated by the coding agent
AI interview	Explanation of the architecture, investigation approach, remediation planning, and evaluation

Do not include `dataset/`, virtual environments, dependency caches, nested archives, API keys, credentials, real personal data, or sensitive configuration in `code.zip`.

6.10 Reasonable Entry Points

There is no required language. If using Python, `code/main.py` is a good entry point. If using another language, document the exact setup and run command clearly in the submitted README.

7. Dataset Development Rules

These rules apply when maintaining or regenerating the challenge dataset itself:

- Generate participant data from private structured truth.
- Keep every person, organization, identifier, and record fictional. Do not use real personal data.
- Use realistic and diverse names, identifiers, request language, timestamps, systems, business records, and evidence formats. Avoid repetitive placeholder patterns that reveal synthetic generation.
- Keep generation deterministic through a fixed seed.
- Preserve referential and temporal consistency across identities, audit history, activity, evidence, sample answers, and hidden golden answers.

- Never hand-edit generated participant data or golden output. Update the private truth or generator logic, regenerate, and validate.
- Do not expose case families, actual statuses, relevant resource lists, expected operations, or golden steps in participant-facing files.
- After generation or contract changes, run:

```
python3 generator/validate_samples.py
python3 generator/validate_dataset.py
```

- Review representative PDFs, screenshots, logs, JSON, text records, and SQL snapshots visually or structurally after regeneration.

8. Cross-Platform And Agent-Compatibility Notes

- Resolve the log path relative to this `AGENTS.md` file, as described in §2. Do not use the platform home directory or hardcode a user path.
- Write logs in UTF-8 with `\n` line endings.
- Do not assume bash. Prefer language-native APIs when possible.
- Keep tool-specific config minimal and point back to this `AGENTS.md`.
- Resolve dataset and output paths relative to the repository root so the solution works on macOS, Linux, and Windows.
- If a nested `AGENTS.md` exists, the closest one wins for files inside that sub-project, but §2 and §5 remain global: keep logging to the `log.txt` beside the top-level `AGENTS.md`, not beside the nested one.

9. Quick Checklist For The Agent

Before responding to any user message, confirm:

- [ ] I have read this file in this session.
- [ ] I have appended the session-start entry.
- [ ] I know that the challenge end time is not configured unless an official source has supplied it.
- [ ] I will append a §5.2 entry after this turn.
- [ ] I have verified that `tool=` exactly matches the harness or coding agent currently running.
- [ ] I have recorded the exact runtime model identifier in `model=`; it is never `unknown`, inferred, or generic.
- [ ] I have confirmed that both the harness name and model name are present and non-empty in the new log entry.
- [ ] I will not log secrets or real personal data.
- [ ] I will use only participant-facing files for predictions.
- [ ] I will preserve the Ghost Data Still Exists deletion-audit and output contract in §6.